MAL2020

Project Title:

Smart Attendance System using Face Recognition

FINAL PROJECT REPORT

Submitted by:

1. Chai Yee Pei BSSE2506028
2. Joey Low Yan Hui BSCS2506056
3. Tan Jun Yee BSCS2506067
4. Terence Lim Chia Mao BSCS2506052

Supervised by:

Mr. Eric Kong Kok Wah

Table of Contents

1.0 Introduction..... 3
2.0 Background..... 5
2.1 Problem Domain and Current Issues..... 5
2.2 Motivation and Importance of the System..... 5
2.3 Legal, Social and Ethical Considerations..... 6
2.4 Professional and Accessibility Considerations..... 6
3.0 Project Management..... 7
3.1 Project Management Approach Adopted..... 7
3.2 Roles and Responsibilities of Team Members..... 9
3.3 Communication and Team Operation..... 11
3.4 Risk Assessment and Management..... 13
4.0 Development Approach..... 14
4.1 System Components..... 15
4.2 Technologies and Tools..... 16
4.3 Data Management..... 17
5.0 Requirements..... 18
5.1 Functional Requirements..... 18
5.2 Non-Functional Requirements..... 20
5.3 System Modelling..... 21
5.3.1 ERD..... 22
5.3.2 Use Case..... 24
6.0 Sprint Planning / Project Scheduling..... 27
6.1 Project Timeline and Phase Planning..... 28
6.2 Task Breakdown, Dependencies and Prioritisation..... 29
6.3 Review, Retrospective and Progress Monitoring..... 30
7.0 Software Engineering..... 31
7.1 Application of Software Engineering Principles..... 31
7.2 Modular and Reusable System Design..... 31

7.3 System Architecture..... 31
7.4 Interaction Between Components..... 32
7.5 Data Flow and Control Mechanisms..... 32
7.6 Class Diagram..... 33
7.7 Sequence Diagram..... 34
7.8 Activity Diagram..... 37
7.9 State Diagram..... 38
7.10 Code Quality and Standards..... 41
8.0 Quality Assurance..... 42
8.1 Quality Assurance Strategy..... 42
8.2 Quality Assurance Checklist..... 42
8.3 Test Plan..... 44
9.0 Conclusion..... 45
Appendix A - Integrated State Diagram of Lecturer and Super Administrator Workflows..... 47
Appendix B - Github Link..... 48
Appendix C - UAT Test Plan..... 49
1.0 Scope..... 49
2.0 Testing Team..... 51
3.0 Milestones and Deliverables..... 52
4.0 Environmental Requirements..... 56
5.0 Features To Be Tested..... 57

1.0 Introduction

Attendance management is an essential administrative process in educational institutions as it allows lecturers and academic administrators to monitor student participation and maintain accurate academic records. Conventional attendance methods such as manual roll calls, sign-in sheets and QR code systems are still widely used. However, these approaches can be time-consuming, prone to human error and vulnerable to misuse, such as proxy attendance. With recent advances in artificial intelligence and biometric technologies, face recognition has emerged as a promising solution for automating identity verification and improving the reliability of attendance tracking systems.

This project introduces a Smart Attendance Face Recognition System, a mobile-based application designed to automate classroom attendance using facial recognition technology. The system enables lecturers to record student attendance by capturing photographs of the classroom using a mobile device. The captured image is then processed through a facial recognition service to detect and identify registered students, after which attendance records are automatically stored in the system database. By replacing traditional manual methods with an automated recognition process, the system aims to reduce administrative workload, minimise attendance fraud and improve overall efficiency in classroom management.

The primary intended users of the system are lecturers, professors and teaching staff responsible for recording attendance during academic sessions. In addition, super administrators serve as system-level managers who oversee platform configuration, user management and system monitoring. Other stakeholders include educational institutions that benefit from accurate attendance data and students whose attendance records are maintained within the system.

The scope of this project focuses on the development of a mobile application integrated with a facial recognition API that enables automated attendance recording. The system includes features such as user authentication, student enrolment with facial image registration, attendance recording through face detection, attendance report generation and system management through role-based access control. The application is implemented using Flutter for the

mobile frontend, Python for the backend services, MySQL for data management and the Face++ API for facial recognition processing. While the system provides automated attendance recording, certain limitations exist. For example, facial recognition accuracy may be affected if students wear items that obstruct their facial features such as masks, sunglasses or other facewear. In such cases, lecturers can manually update attendance records through the application.

The development of this system is motivated by the increasing adoption of biometric technologies in various industries such as security, access control and identity verification. Despite the widespread use of facial recognition in many sectors, it's application in attendance management within colleges in the northern region remains relatively limited. This project therefore explores the potential of integrating modern biometric technologies into educational environments to improve administrative processes and support digital transformation in academic institutions.

This report documents the design, development and evaluation of the Smart Attendance Face Recognition System. The report is organised into several sections. Section 2 discusses the background of the problem domain and the motivation behind the system. Section 3 presents the project management approach and team responsibilities. Section 4 describes the system development methodology and architecture, followed by system requirements and modelling in Section 5. Section 6 outlines project scheduling and sprint planning, while Section 7 explains the application of software engineering principles in the system design. Section 8 discusses the quality assurance strategy used to ensure system reliability and performance. Finally, Section 9 concludes the report by summarising the achievements of the project and discussing potential future improvements.

2.0 Background

2.1 Problem Domain and Current Issues

Attendance management plays an important role in the education system because it helps institutions monitor student participation and maintain academic discipline. In classroom environments, lecturers need to record attendance while also managing teaching activities. Traditional methods such as manual roll calls, sign-in sheets, and QR code scanning are still commonly used, but they can interrupt lessons and take up valuable teaching time (Anjum et al., 2023). With the development of mobile technology and biometric systems, face recognition technology has become a possible solution for automating identity verification. This project focuses on developing a mobile-based attendance system that allows lecturers to quickly record attendance using face recognition in a classroom environment.

Current attendance methods have several limitations that affect their effectiveness. Manual roll calls and sign-in sheets are time-consuming and prone to human errors such as missing or incorrect entries. QR code attendance systems are faster but can be misused when students share the code with absent friends. RFID or smart card systems require additional hardware and cards can be lost, stolen, or exchanged between students. Because these methods cannot reliably verify a student's identity, they may lead to proxy attendance and inaccurate records.

2.2 Motivation and Importance of the System

The main motivation for developing this system is to improve the efficiency and accuracy of attendance recording in educational institutions. Many existing methods are outdated and allow attendance fraud or errors to occur. By using face recognition technology in a mobile application, the system can automatically verify student identities and record attendance more reliably. This reduces manual work for lecturers and saves classroom time. In addition, the project aims to apply modern technology to solve real-world problems in the education sector.

This system is important because it provides a faster and more accurate way to record attendance. Face recognition ensures that attendance is recorded only for students who are physically present in class, reducing the risk of proxy attendance. The system also helps lecturers save time and automatically generates attendance records. Educational institutions benefit from improved data accuracy and better attendance monitoring. Overall, the system supports digital transformation in education and improves the efficiency of attendance management.

2.3 Legal, Social and Ethical Considerations

The system follows relevant laws and regulations related to data protection and privacy. It complies with the Personal Data Protection Act (PDPA) 2010 in Malaysia to ensure that personal data such as facial images and student information are collected and used only for attendance purposes. User consent is obtained before storing facial data, and access to information is restricted to authorised users. The project also respects copyright laws and follows proper licensing when using third-party services such as the Face++ API. Security measures are implemented to prevent unauthorised access and data misuse.

Social and ethical considerations are important when using facial recognition technology. The system ensures that facial data is used only for attendance verification and not for surveillance. User privacy is protected through secure data storage and restricted access. The system also treats all users fairly and does not discriminate against any group of students. By informing users about how their data is collected and used, the system promotes transparency and responsible use of technology.

2.4 Professional and Accessibility Considerations

The development of the system follows professional standards in software engineering. The project team ensures that the system is built using proper coding practices, system design, and documentation. Regular testing is carried out to maintain quality, reliability, and security. Each team member is responsible for completing their assigned tasks and maintaining project progress. Following these professional practices helps ensure the system is developed responsibly and meets expected software quality standards.

The system is designed to be simple and easy to use for all users. The mobile application provides a clear interface, readable text, and simple navigation so that lecturers can operate it without difficulty. The system is compatible with common smartphones and does not require advanced technical knowledge. By focusing on usability and inclusive design, the system ensures that users with different technical abilities can use it effectively.

3.0 Project Management

3.1 Project Management Approach Adopted

The Smart Attendance Face Recognition project adopted the Agile development methodology to ensure flexible planning, iterative development, and continuous monitoring throughout the project lifecycle. Agile was selected because the system consists of multiple interdependent components, including facial recognition, attendance processing, database integration, and role-based access control. These components required continuous testing, refinement, and adjustment to achieve optimal performance and accuracy.

Agile was particularly suitable for this project due to its moderate size and technical complexity (Coursera Staff, 2023). Since facial recognition accuracy and system integration could not be fully predicted at the initial stage, a flexible and adaptive approach was necessary. Agile allowed the team to respond quickly to technical challenges, modify system features based on testing outcomes, and progressively improve performance. Additionally, the team's familiarity with iterative development practices made Agile an appropriate and manageable choice.

The project was structured into short iterative cycles (sprints). Each sprint includes key events such as sprint planning, development, testing, review, and retrospective evaluation. During sprint planning sessions, tasks were identified, prioritised, and assigned to team members based on their roles and expertise (Schwaber and Sutherland, 2020). This approach enabled system features to be implemented incrementally and refined based on testing outcomes and observed system performance. A task list was created for each sprint to ensure clarity of responsibilities and deadlines.

To efficiently manage activities and timeframes, a module-based development method was used. The system was divided into functional modules, each allocated to specific sprints depending on complexity and dependencies. Core modules such as user management and role-based access control were developed first, followed by attendance logging using face recognition. Once these primary modules were stable, supporting features such as reporting, dashboard interfaces, and data

management were implemented. This phased arrangement ensured that essential functionalities were completed and tested before dependent features were introduced.

At the end of each sprint, deliverables were reviewed collectively to evaluate progress and system functionality. Each iteration produced measurable outputs, including completed modules, integrated components, and tested features. Continuous testing activities, including unit testing, integration testing, and usability evaluation, were conducted within each sprint to validate system reliability before proceeding to the next stage.

Overall, the Agile methodology provided a structured yet flexible framework that supported progressive feature development, early issue detection, and continuous improvement. This approach ensured that the project remained aligned with its objectives, was delivered on schedule, and maintained high standards of system quality and reliability.

3.2 Roles and Responsibilities of Team Members

This project was executed through collaborative teamwork, with each member assigned distinct tasks and duties based on their distinctive skills and knowledge. Clear role allocation guaranteed accountability, effective task execution, and easy cooperation throughout the Smart Attendance Face Recognition system development process.

Team Leader and Programmer – Terence Lim Chia Mao

The team leader and programmer were in charge of monitoring overall project progress and ensuring that tasks were accomplished within the timeframe specified. This involved arranging team meetings, assigning work, monitoring milestones, and staying on track with project goals. As a programmer, he helped with system development by adding Google and Microsoft login authentication via OAuth 2.0, as well as contributing to API testing, debugging, and feature refinement. His responsibilities included both successful project coordination and secure user authentication.

Programmer and Analyst – Chai Yee Pei

The programmer and analyst focused on backend development and system analysis. The responsibilities included designing and implementing facial recognition technology with the Face++ API, creating the database structure, and connecting backend services with the mobile application. As an analyst, this role involved analysing system workflows, defining data flows, evaluating technical constraints, and contributing to system architecture planning. This role ensured stable backend operations, accurate attendance processing, and well-structured system logic.

Researcher and Programmer – Joey Low Yan Hui

The researcher and programmer were in charge of performing research on facial recognition technologies, attendance systems, and user interface design trends. The research findings aided educated decision-making on API selection, feature design, and usability enhancements. As a programmer, this position entailed creating essential mobile application functionality, integrating backend logic, and testing and

debugging. This position helped to improve both the technical reliability of the system and the relevance of design decisions.

Designer and Programmer – Tan Jun Yee

The Designer and Programmer were in charge of designing the user interface and user experience, as well as developing the frontend. The responsibilities included developing wireframes and prototypes, building responsive layouts, and translating designs into working Flutter code. This role also required creating reusable UI components, integrating visual data representations, and assuring accessibility and usability. Collaboration with backend developers enabled seamless API integration and uniform data display.

3.3 Communication and Team Operation

Effective communication and collaboration were essential for smooth teamwork and the timely completion of the project. The team adopted a structured approach that combined messaging, meetings, file-sharing, version control, and task coordination tools to support task allocation, progress monitoring, documentation, and overall project management.

Project tasks were assigned according to team members' roles, expertise, and responsibilities. Core development tasks, including backend programming, frontend implementation, and API integration, were allocated based on technical strengths, while research, design, and documentation tasks were clearly distributed to maintain a balanced workload. Progress was tracked using GitHub for code management and commit history, supplemented by shared sprint task lists and updates during meetings and messaging platforms. This allowed the team to monitor task completion, identify potential delays, and ensure deadlines were consistently met.

Google Drive and Google Docs were used as the primary platforms for document creation, storage, and collaboration. Project materials, including proposals, system design documents, reports, and diagrams, were centrally stored in structured shared folders organised by document type, ensuring systematic organisation and easy retrieval. Team members contributed content collaboratively in Google Docs, with editing access to update content, provide comments, and track revisions in real time. Major structural changes were discussed during weekly meetings before implementation to maintain consistency and document integrity. Canva was used to design and share presentation slides collaboratively, while PebblePad served as the academic documentation and submission platform, supporting reflective documentation, evidence collection, and structured project submission. Important updates, technical decisions, and documentation changes were communicated through WhatsApp and further clarified during face-to-face meetings to ensure alignment among team members.

Face-to-face meetings were held weekly throughout the project, with extra ad hoc meetings organised as needed to address urgent technical challenges or development issues. These meetings were semi-formal, following a prepared

agenda that included progress updates, task reviews, technical problem resolution, and planning for upcoming sprint activities. During meetings, team members reported on completed tasks, ongoing work, and any challenges encountered. Sprint reviews assessed module completion, evaluated system functionality, and determined readiness for integration or testing.

When issues arose, they were immediately communicated to the team through WhatsApp or during meetings. The team followed a collaborative problem-solving approach, discussing the issue, proposing solutions, and assigning responsibility for implementation. Technical problems, such as API errors or debugging challenges, were typically addressed by the member most familiar with the relevant module, while critical decisions were made collectively to ensure accuracy and prevent delays.

Team members actively supported one another throughout the project. Collaboration practices included peer assistance in coding, joint debugging sessions, feedback on design, and knowledge sharing regarding API integration and system architecture. Open communication channels allowed members to clarify doubts, exchange ideas, and improve system functionality collectively. Regular in-person discussions and real-time messaging facilitated smooth coordination, ensured alignment with project goals, and strengthened the overall quality and reliability of the system.

The integration of structured documentation, communication tools, task tracking, and collaborative practices ensured transparency, efficiency, and accountability across all project activities, contributing significantly to the project’s successful completion.

3.4 Risk Assessment and Management

Effective risk management was essential to ensure the successful completion of the project. Several potential risks were identified during the planning and execution phases. Time constraints posed a significant challenge, as delays could arise from the complexity of implementing facial recognition or from unforeseen technical issues. Resource limitations were considered a potential risk, particularly because testing had to be conducted across different real smartphones, each with varying performance, screen sizes, and system capabilities (Coursera Staff, 2025). These variations could affect the accuracy and consistency of system testing, particularly for facial recognition and app responsiveness. Technical challenges were another consideration, including integration difficulties with the Face++ API, backend-server communication issues, or errors in facial recognition that could compromise the reliability of attendance tracking. In addition, team coordination risks, such as miscommunication or unclear task assignments, could result in overlapping work or missed deadlines. Finally, data security and privacy risks were identified due to the sensitive nature of storing and processing student data and facial images.

Each identified risk was evaluated based on its likelihood of occurrence and potential impact. High-priority risks, such as technical challenges and time constraints, were considered most critical because they could significantly delay project progress or affect system functionality. Medium-priority risks, including resource limitations and team coordination issues, were managed proactively to prevent disruption to development. Even lower-probability risks, such as data security concerns, were addressed due to their potential high impact. This assessment helped the team prioritise risk management efforts and focus resources on the most critical areas.

To mitigate these risks, several strategies were implemented. Time constraints were addressed by organising development into clearly defined modules and short iterative sprints, with progress reviewed at the end of each sprint to identify delays early. Resource limitations were addressed by testing the system across multiple real smartphones to ensure consistent performance, accuracy, and reliability across different devices. Technical challenges were minimised through thorough API research, continuous integration testing, and collaborative problem-solving among

team members. Team coordination risks were reduced by holding weekly face-to-face meetings and maintaining daily communication via WhatsApp to clarify tasks and monitor progress. Data security and privacy were ensured through encryption of sensitive information, secure database storage, and controlled access based on team roles.

These proactive risk mitigation and contingency measures ensured that potential issues were monitored, reduced, and effectively managed, allowing the project to proceed smoothly and achieve its objectives within the planned timeline.

4.0 Development Approach

The system was developed using a structured, iterative, and modular approach to ensure reliability, usability, and maintainability. The project followed a module-based development method, where the system was divided into functional components such as user management, attendance logging, database management, reporting, and dashboard interfaces. Each module was designed, implemented, tested, and integrated individually, which allowed the team to focus on building stable and functional components before integrating them into the complete system.

Design activities began with system architecture planning, including user interface layout, data flows, API interactions, and interactions between the frontend, backend, and database. Implementation was conducted using Flutter for the mobile frontend and Python for the backend, with integration of the Face++ API for facial recognition. Each module underwent iterative unit testing to ensure functionality, followed by integration testing to validate interactions between system components. This structured workflow allowed for continuous monitoring, early error detection, and progressive system improvement throughout the development lifecycle.

The choice of Agile methodology supported iterative development, flexibility, and timely feedback. Tasks were divided into sprints, with clear objectives, deliverables, and review processes. Regular sprint reviews and testing cycles enabled the team to refine features progressively, particularly accuracy-sensitive components such as facial recognition (Coursera Staff, 2023).

Technical decisions were guided by project requirements, system performance, scalability, and maintainability. Flutter was selected for mobile app development due to its cross-platform capabilities, fast rendering, responsive design features, and efficient integration with backend APIs. Python was chosen for backend development for its flexibility, extensive libraries, and compatibility with RESTful APIs like Face++. MySQL was used as the relational database to manage structured attendance data securely and efficiently. GitHub was adopted for version control, supporting collaborative development, change tracking, and structured code management. These technologies collectively enabled efficient development, high maintainability, and reliable system performance.

4.1 System Components

The system was developed with a focus on simplicity, usability, and efficiency. The user interface was designed to support both lecturers and superadmins, providing role-specific screens and functionalities. Lecturers could log in, access their dashboard, enrol students, take attendance, view reports, and configure personal settings, while superadmins had access to a dashboard, view system logs, manage the list of administrators, and configure system-wide settings. Navigation was structured logically to guide users through the system seamlessly, while responsive layouts, clear notifications, and informative error messages ensured accessibility and ease of use across devices with varying screen sizes.

All system processing was handled by the backend, which managed tasks such as face recognition, attendance recording, and data validation. Workflows were implemented to process face images, detect and identify students using the Face++ API, and store attendance results in the database. Business logic also enforced role-based access, guaranteeing that lecturers and administrators could access only the features relevant to their responsibilities.

A relational database (MySQL) was used to store user profiles, facial data, attendance records, and system logs. The database was structured with carefully defined relationships between students, classes, and attendance sessions to ensure data consistency and integrity. Constraints and indexing were applied to maintain accuracy, optimise query performance, and support efficient retrieval of attendance reports.

Integration between components ensured smooth data flow across the system. The mobile frontend communicated with the backend via RESTful API calls, transmitting user inputs and receiving processed results. The backend processed these requests, interacted with the database, and returned responses for display in the UI. This seamless integration allowed attendance data captured through facial recognition to be accurately reflected in real time on dashboards and reports, supporting reliable monitoring and management of attendance records.

4.2 Technologies and Tools

Programming Languages

- **Flutter (Dart):** Chosen for mobile frontend development due to cross-platform support and efficient UI rendering.
- **Python:** Selected for backend development for ease of API integration, rapid development, and extensive library support.

Frameworks and Libraries

- **Flutter SDK:** Streamlined UI development and ensured consistent layouts across devices.
- **Face++ API:** Provided reliable and accurate face detection and recognition capabilities.
- **Flask / FastAPI:** Enabled fast backend API development and request handling.

Development Tools

- **Visual Studio Code:** Used for coding, debugging, and compiling frontend and backend components.
- **GitHub:** Managed source code version control, collaboration, and issue tracking.

Testing and Deployment Tools

- **Unit Testing & Integration Testing Tools:** Ensured functionality of modules and system-wide integration.
- **Emulators and Real Devices:** Used to verify mobile app performance, responsiveness, and accuracy.
- **Deployment Platform:** Backend and database hosted on a server environment for system testing and live usage.

4.3 Data Management

The system employed a relational database (MySQL) to store and manage structured data. Core entities included users (lecturers and superadmins), students, subjects, classes, student faces, enrollments, attendance records, and system logs. Relationships between tables were carefully defined using foreign keys to maintain referential integrity. For example, subjects were linked to lecturers, classes were linked to subjects, and enrollments connected students to subjects. The database design supported multiple facial images per student via the student_faces table, ensuring flexible face recognition and accurate attendance tracking.

Data integrity and consistency were maintained through backend validation, unique constraints, and structured workflows. The team collaborated to verify database entries, share knowledge on data relationships, and perform peer reviews for critical updates. Regular testing and backups ensured the data remained accurate, consistent, and recoverable throughout the project lifecycle.

Role-based access control was enforced at the application level to protect sensitive data. Only authorised users could view or modify records according to their roles, with superadmins managing overall system settings and lecturers accessing subject and attendance data relevant to their classes. Authentication was implemented using Google and Microsoft OAuth 2.0, while sensitive information, including facial images and user credentials, was securely stored and encrypted. Data transmission between the frontend and backend was also encrypted, ensuring confidentiality and protecting student privacy.

This structured data management approach provided a reliable, secure, and scalable foundation for the system, allowing accurate attendance tracking, efficient data retrieval, and seamless integration between the UI, backend, and reporting modules.

5.0 Requirements

5.1 Functional Requirements

Functional requirements describe the core functions that the Smart Attendance Face Recognition System must perform in order to meet user needs. These requirements define the interactions between users and the system and outline the specific services provided by the application.

The system supports two main user roles, which are Lecturers and Super Administrator. Each role is provided with different functionalities based on their responsibilities.

Admin Functionalities

Lecturers are responsible for managing student attendance within their classes. The system provides the following functions:

Table 5.0: Functional Requirements for Lecturers

ID	Functional Requirement	Description
FR01	User Login	Lecturers to login using email and password or OAuth authentication through Google or Micorsoft accounts.
FR02	Dashboard Access	After login, lecturers can view a dashboard that displays subject, class information, and quick access to system features.
FR03	Manage Subject and Class	Lecturers can create subjects and manage classes associated with those subjects.
FR04	Student Enrollment	Lecturers can register students and store facial data.

FR05	Face Recognition Attendance	During a class session, lecturers can capture images using the mobile application camera. The system automatically detects and recognises student faces using the Face++ API and records attendance.
FR06	Attendance Record Management	Lecturers can view and manage attendance records for their classes.
FR07	Report Generation	Lecturers can generates attendance reports that can be viewed or exported as PDF files for record keeping.
FR08	User Settings	Lecturers can modify their personal account settings.

Superadmins Functionalities

Superadmins manage system-wide configurations and administrative tasks.

Table 5.1: Functional Requirements for Super Administrators

ID	Functional Requirement	Description
FR01	Admin Management	Superadmins can manage administrator accounts within the system.
FR02	System Logs Monitoring	Superadmins can monitor system activity.
FR03	System Configuration	Super administrators can configure system-wide settings such as authentication services or platform parameters.

5.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the Smart Attendance Face Recognition System. The following requirements ensure that the system operates efficiently, securely, and reliably during daily usage.

Performance

The system should process facial recognition requests efficiently to avoid disrupting classroom activities. Attendance verification should ideally be completed within 4 seconds for 1 person, 12 seconds for 5 person, 19 seconds for 10 person and 27 seconds for 15 person per recognition.

Security

Security is critical because the system handles sensitive data such as student information and facial images. The system implements several security measures, including encrypted data storage, secure API communication, and role-based access control to prevent unauthorised access.

Reliability

The system must consistently record attendance accurately and maintain stable communication between the mobile application, backend server, and facial recognition service.

Usability

The mobile application interface should be simple and intuitive so that lecturers can use the system easily without requiring technical expertise. Clear navigation, readable text, and responsive layouts ensure a user-friendly experience.

Scalability

The system should be capable of handling increasing numbers of users, students, and attendance records without affecting system performance.

Compatibility

The application should run smoothly on common smartphone devices and support multiple screen sizes and operating system versions.

Maintainability

The system is developed using modular architecture and clear coding standards to ensure that future developers can maintain or extend the system easily.

5.3 System Modelling

System modelling provides a visual representation of the structure and functionality of the Smart Attendance Face Recognition System. These models help stakeholders and developers understand how data is organised within the system and how users interact with the application before the actual implementation process.

In this project, system modelling focuses on two key diagrams that support system design and requirement analysis:

- **Entity Relationship Diagram (ERD)** – illustrates the structure of the database, including entities, attributes, and relationships used to store and manage system data.
- **Use Case Diagram** – describes the interactions between system users and the system functionalities. It identifies the actors involved and the operations they can perform within the system.

5.3.1 ERD

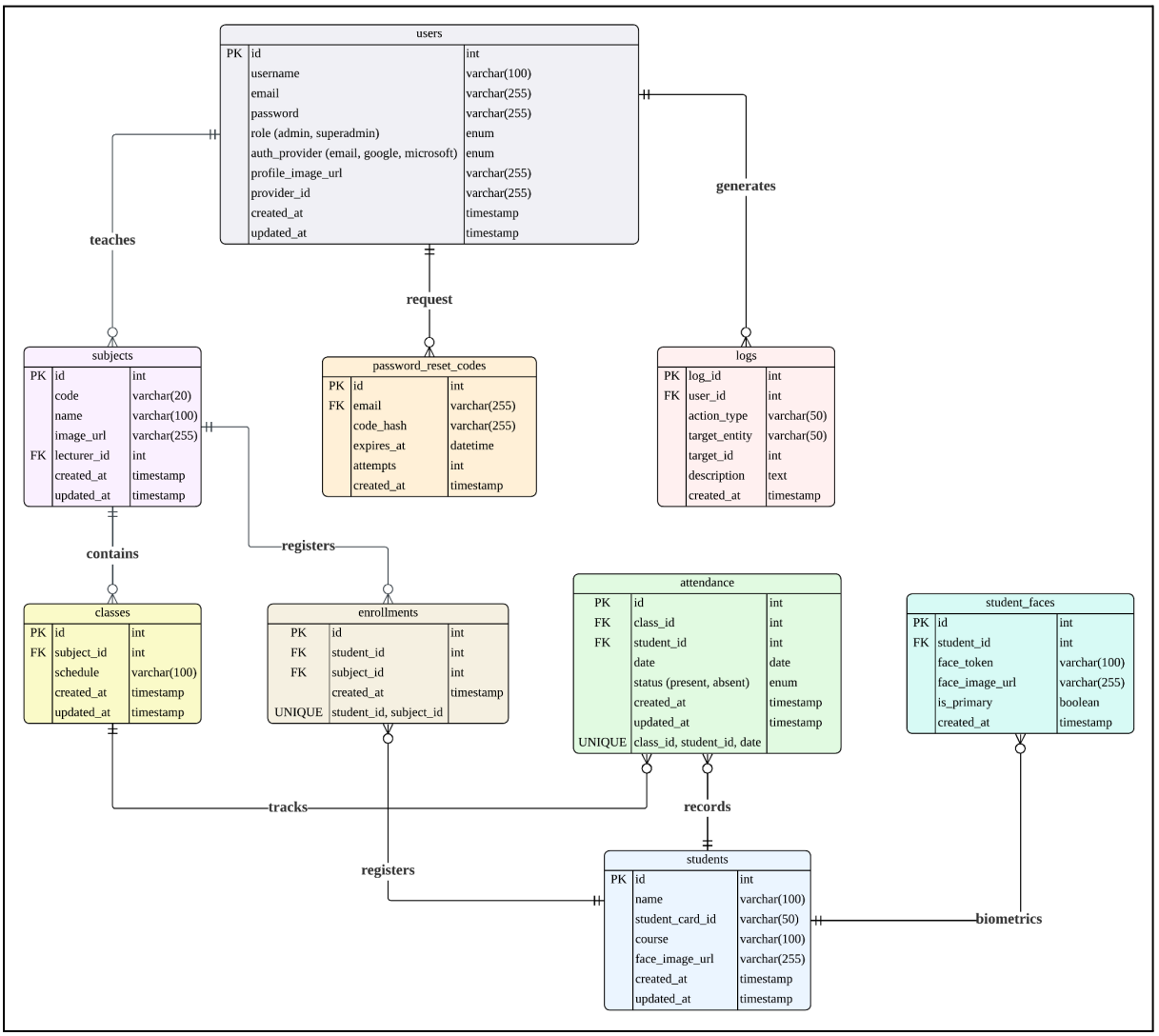

Figure 5.0 Entity Relationship Diagram (ERD) of the Smart Attendance Face Recognition System

Figure 5.0 above represents the database architecture of the Smart Attendance Face Recognition System. It defines the entities, attributes, and logical constraints required to maintain data integrity and support the automated attendance workflow.

Core Entities and Relationships

- **Users:** This entity serves as the primary authentication table for lecturers and super-administrators. It stores unique credentials, including username, email, and hashed password. It also manages auth_provider details (Email, Google,

or Microsoft) and user roles. A one-to-many (1:N) relationship exists between Users and Subjects, as one lecturer can manage multiple academic courses.

- **Students:** This entity stores personal profiles, including name, student_card_id, and course. To support the biometric requirements of the system, it has a one-to-many (1:N) relationship with the Student_faces entity, allowing multiple facial tokens and images to be stored for a single student to improve recognition accuracy.
- **Subjects & Classes:** The Subjects entity represents the academic modules. Each subject is linked to the Classes entity via a one-to-many (1:N) relationship, where each class record represents a specific scheduled session (a specific time or date) within that subject's curriculum.
- **Enrollments:** This functions as a junction table to resolve the many-to-many (M:N) relationship between Students and Subjects. It ensures that a student can be registered for multiple courses and each course can accommodate many students.
- **Attendance:** This is the central transaction entity. It links Students and Classes to record the status (Present or Absent) and date of attendance. The relationship is structured so that each record uniquely identifies a student's presence in a specific class session.

Security and System Management

- **Password_reset_codes:** This entity manages account recovery security. It is linked to the Users entity via the email attribute. It stores temporary hashed verification codes, expiration timestamps and includes a security threshold of 5 failed attempts to prevent brute-force attacks.
- **Logs:** To ensure system accountability, the Logs entity records all administrative activities. Every action is tied to a specific User, providing an audit trail that includes the action_type, target_entity, and a detailed description of the event.

5.3.2 Use Case

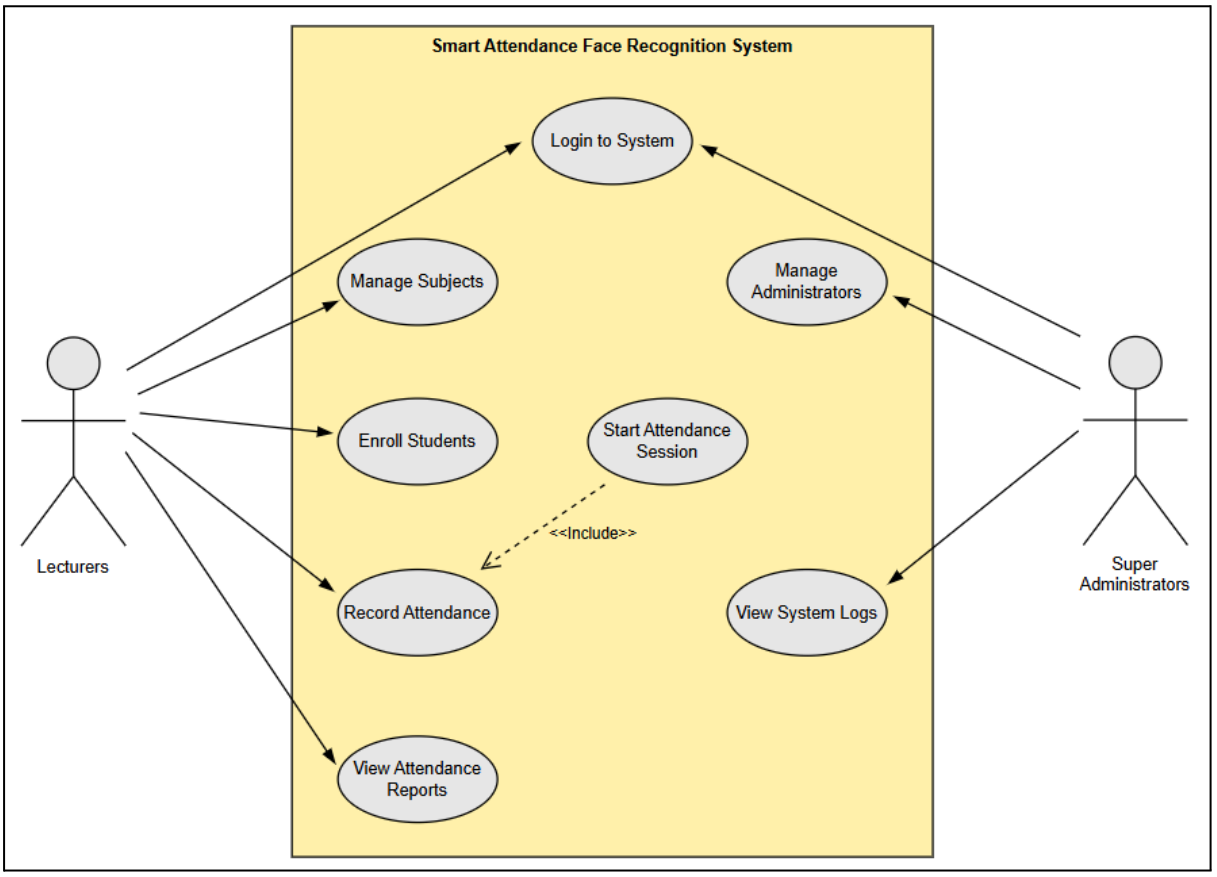

Figure 5.1: Use Case Diagram of the Smart Attendance Face Recongnition System

Figure 5.1 above illustrates the interactions between system users and the Smart Attendance Face Recognition System. The two main actors in the system are Lecturers and Super Administrators. Lecturers perform operations such as managing subjects, enrolling students, starting attendance sessions, recording attendance using facial recognition, and viewing attendance reports. Super administrators are responsible for managing administrator accounts and monitoring system logs.

Table 5.2: Use Case Descriptions for Smart Attendance Face Recognition System

ID	Name	Actor	Description	Precondition	Main Flow	Postcondition
UC01	Login to System	Lecturers, Super Administrators	User logs into the mobile application using email/password or OAuth authentication (Google/Microsoft).	User has a valid account.	1. User opens the application. 2. User enters login credentials or selects OAuth login. 3. System verifies credentials. 4. User is granted access to the dashboard.	User successfully logged into the system.
UC02	Manage Subjects	Lecturers	Lecturer creates and manages subjects for teaching.	Lecturer logged in.	1. Lecturer Enter "Subjects". 2. Lecturer adds or edits subject details. 3. System stores subject information.	Subject information saved in database.
UC03	Enroll Students	Lecturers	Lecturer registers students and uploads their facial	Lecturer logged in and subject exists.	1. Lecturer enroll each of the student. 2. Lecturer enters student information.	Student successfully registered in system.

			images for recognition.		3. System stores student and face data.	
UC04	Start Attendance Session	Lecturers	Lecturer initiates an attendance session for a class.	Lecturer logged in and class exists.	1. Lecturer opens attendance module. 2. Lecturer selects subject/class. 3. Lecturer starts attendance session.	Attendance session activated.
UC05	Record Attendance	Lecturers	System captures classroom image and performs face recognition to record student attendance.	Attendance session started and students enrolled.	1. Lecturer captures classroom photo. 2. System sends image to backend. 3. Face++ API performs recognition. 4. System matches faces with student database. 5. Attendance is recorded automatically.	Attendance stored in database.

UC06	View Attendance Reports	Lecturers	Lecturer views attendance records and generates reports.	Lecturer logged in.	1. Lecturer selects report module. 2. System retrieves attendance records. 3. Lecturer views or exports report (PDF).	Attendance report generated.
UC07	Manage Administrators	Super Administrators	Super administrator manages system administrators.	Super Administrator logged in.	1. Super admininstrator opens admin management page. 2. Add/edit/remove administrator accounts. 3. System updates records.	Administrator list updated.
UC08	View System Logs	Super Administrators	Super administrator monitors system activities through logs.	Super Administrator logged in.	1. Super admininstrator opens system logs page. 2. System displays activity records.	System activities monitored.

The use case description explains the main interactions between system users and the Smart Attendance Face Recognition System. It identifies the actors involved in the system and the operations they are able to perform within the application. The two primary actors in the system are Lecturers and Super Administrators.

Lecturers interact with the system to manage subjects, enrol students, initiate attendance sessions, record attendance using the facial recognition feature, and generate attendance reports. Super administrators have higher-level privileges and are responsible for managing administrator accounts and monitoring system activities through system logs.

Each use case describes the preconditions required before the operation can be performed, the sequence of actions involved in completing the task, and the resulting system state after the operation is completed. This structured representation helps clarify system functionality and ensures that all user interactions are clearly defined during the system design and development process.

5.3.3 CRUD Diagram

Table 5.3: CRUD Matrix for Smart Attendance Face Recognition System

System Function/ Module	Users	Students	Subjects	Classes	Enrollments	Attendance	Student Faces	Logs
User Login	R	-	-	-	-	-	-	C
Manage Users (Superadmin)	C/R/U/D	-	-	-	-	-	-	C
Create Subject	-	-	C	-	-	-	-	C

Manage Subjects	-	-	R/U/D	-	-	-	-	C
Create Class	-	-	R	C	-	-	-	C
Enrol Students	-	C	R	-	C	-	C	C
Capture Attendance	-	R	-	R	R	C	R	C

Manage Attendance Records	-	R	-	R	-	R/U/D	-	C
Generate Reports	-	R	R	R	-	R	-	-
View System Logs	-	-	-	-	-	-	-	R
Password Reset	R	-	-	-	-	-	-	-

Table 5.3 illustrates the interactions between system functionalities and database entities within the Smart Attendance Face Recognition System. It identifies how each module performs Create(C), Read(R), Update(U), and Delete(D) operations on specific database tables. This representation helps clarify data flow, system responsibilities and the relationships between application

features and stored data. By mapping system functions to database operations, the CRUD matrix ensures that all data interactions are well defined, consistent and aligned with system responsibilities.

6.0 Sprint Planning / Project Scheduling

WBS NUMBER	TASK TITLE	START DATE	DUE DATE	DURATION	PCT OF TASK COMPLETE
1	**Project Management**				
1.1	Requirement gathering & proposal submission	22/09/25	05/12/25	73	100%
1.2	Literature review & project study	07/10/25	20/10/25	13	100%
2	**System Design & Requirements**				
2.1	System design (architecture, database)	21/10/25	03/11/25	12	100%
3	**User Interface (UI) Development**				
3.1	Initial mobile app interface	04/11/25	17/11/25	13	100%
4	**Backend / Communication Systems**				
4.1	Functional backend & database connectivity	18/11/25	07/03/26	109	100%
5	**API Development**				
5.1	Working face recognition module	02/12/25	26/02/26	84	100%
5.2	Further research & catch-up	16/12/25	09/01/26	23	100%
6	**System Integration**				
6.1	Fully integrated prototype	30/12/25	13/03/26	73	100%
7	**Testing & Validation**				
7.1	Testing & evaluation report	13/01/26	02/03/26	49	100%
7.2	Stable & improved project version	12/01/26	15/03/26	63	100%
8	**Documentation & Final Delivery**				
8.1	Drafted final report	10/02/26	23/02/26	13	100%
8.2	Completed final report & slides	24/02/26	16/03/26	22	100%
8.3	Submission & wrap-up	17/03/26	01/04/26	14	100%

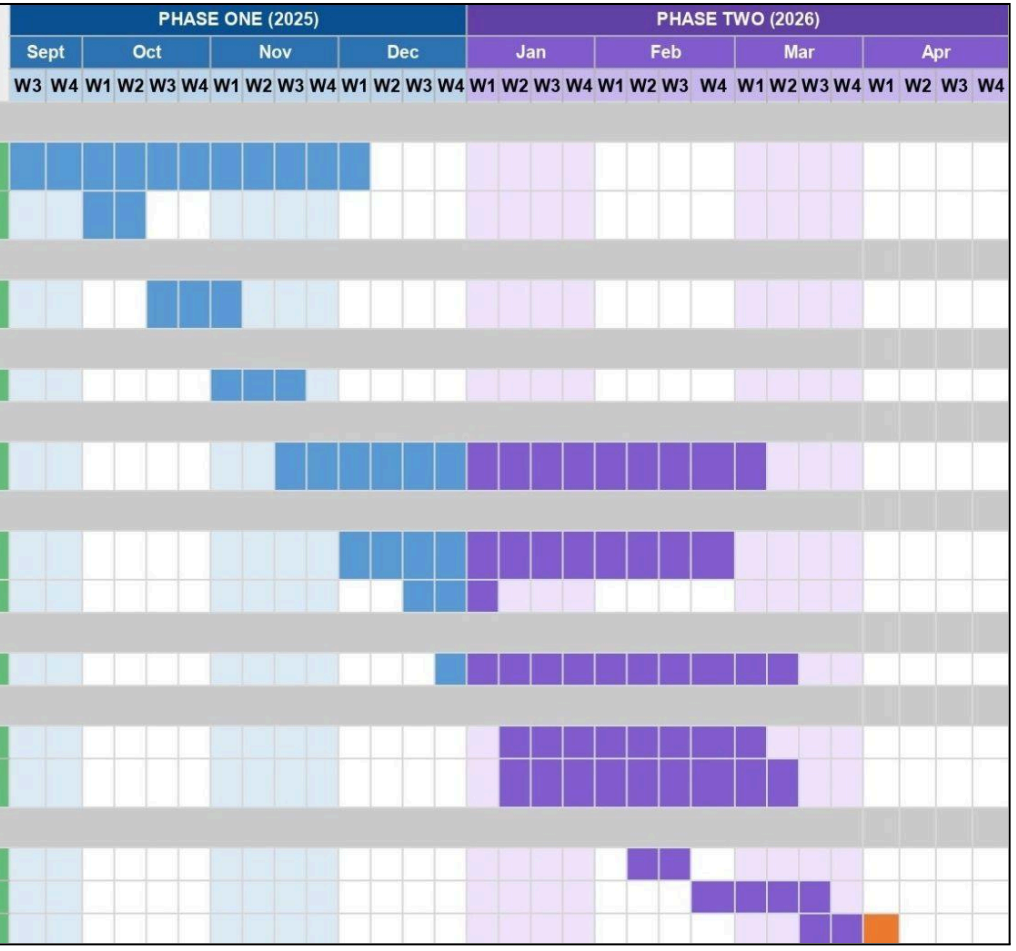

Figure 6.0: Sprint Planning and Project Scheduling Timeline

6.1 Project Timeline and Phase Planning

The project was structured according to a clearly defined timeline, as illustrated in the Gantt chart. Development commenced with project management activities, including requirement gathering and proposal submission (22/09/25 – 05/12/25), alongside literature review and project study (07/10/25 – 20/10/25). These initial activities were essential in establishing a clear understanding of system requirements, technical feasibility, and overall project objectives. By completing these foundational tasks first, the project reduced the risk of scope ambiguity and ensured alignment with academic and functional expectations.

Following the planning stage, the project progressed through structured development phases rather than isolated short sprints. The System Design & Requirements phase (21/10/25 – 03/11/25) focused on defining the overall system architecture, database schema, and technical framework. This provided a blueprint that guided subsequent implementation work. The User Interface Development phase (04/11/25 – 17/11/25) then concentrated on creating the initial mobile application interface, ensuring usability and logical navigation before backend integration.

The Backend / Communication Systems phase (18/11/25 – 07/03/26) focused on implementing server-side logic, API endpoints, and database connectivity. In parallel, the API Development phase (02/12/25 – 26/02/26) involved further research on facial recognition technologies, integrating the Face++ API, and configuring facial detection and matching processes to enable automated attendance recording. This phase also included technical exploration, testing, and catch-up activities to address implementation challenges and ensure stable performance before full system integration. Once these components were individually functional, the System Integration phase (30/12/25 – 13/03/26) combined the UI, backend, and AI modules into a cohesive prototype.

The final stages included Testing & Validation (13/01/26 – 15/03/26), where system functionality, reliability, and performance were evaluated, followed by Documentation & Final Delivery (10/02/26 – 01/04/26), culminating in submission and project wrap-up. Key milestones such as system design completion, backend functionality, face recognition integration, integrated prototype delivery, and final documentation

submission served as measurable checkpoints to track progress and maintain development discipline.

6.2 Task Breakdown, Dependencies and Prioritisation

To ensure efficient execution, each major phase was divided into smaller, clearly defined tasks. This task decomposition improved accountability and allowed progress to be measured incrementally. For example, database schema design was broken down into table creation, relationship definition, constraint implementation, and testing. Similarly, backend development included authentication implementation, API creation, attendance processing logic, and data validation mechanisms.

Task dependencies were carefully identified to maintain a logical workflow. The database structure had to be fully defined before backend APIs could interact with stored data. Backend services needed to be stable before integrating the face recognition module for attendance processing. Furthermore, the Face++ facial recognition API required appropriate database structures for storing facial images and tokens before it could reliably perform identity detection and matching. System integration depended on the completion and stability of the UI, backend services, and facial recognition functionality. Testing activities were conducted only after integration to ensure that all modules worked cohesively under realistic usage scenarios.

Priority setting played a critical role in maintaining focus and stability. Core functionalities such as authentication, database management, backend logic, and face recognition integration were prioritised as they formed the foundation of the Smart Attendance system. Secondary tasks, including optimisation, interface refinements, and documentation polishing, were scheduled after essential system components were operational. This prioritisation strategy ensured that the system remained functional even if minor enhancements required adjustment.

6.3 Review, Retrospective and Progress Monitoring

Continuous evaluation was incorporated throughout the project lifecycle to ensure alignment with planned objectives. After each major phase, progress reviews were conducted to verify deliverables against requirements. Functional modules such as the initial UI prototype, backend connectivity, and face recognition implementation were tested and demonstrated to confirm operational readiness. These reviews allowed early detection of issues and reduced the risk of compounded technical problems in later stages.

In addition to technical reviews, reflective discussions were carried out to assess team coordination, time management, and problem-solving effectiveness. Challenges encountered during development, such as integration complexity or debugging delays, were analysed to identify process improvements. Lessons learned from each phase informed adjustments in task allocation and scheduling for subsequent stages.

Progress monitoring was conducted by comparing planned timelines in the Gantt chart with actual completion dates. This enabled early identification of minor delays and allowed corrective actions, such as reallocating effort or adjusting task sequencing. The visual representation of overlapping tasks and dependencies in the Gantt chart supported informed decision-making and improved time management. Through systematic monitoring and periodic evaluation, the project maintained steady progress toward final submission while ensuring the quality and completeness of deliverables.

7.0 Software Engineering

7.1 Application of Software Engineering Principles

The Smart Attendance System was developed with fundamental software engineering principles to ensure maintainability, scalability, reliability, and security. Established coding standards were adhered to, promoting clarity and consistency, while GitHub was employed for version control to facilitate collaboration and track changes. System reliability was reinforced through iterative testing of individual modules, and security was ensured through role-based access control, encrypted facial data storage, and OAuth 2.0 authentication. Together, these measures ensure that the system remains robust, maintainable, and adaptable to future requirements.

7.2 Modular and Reusable System Design

The system is structured into independent modules, each responsible for specific functions such as authentication, enrolment, attendance logging, reporting, and super administrator management. This modular architecture enables system components to be reused in other systems, thereby reducing development time and effort. For instance, the authentication module, which utilises Google and Microsoft Single Sign-On (SSO), can be integrated into other academic platforms. This design enhances system scalability and simplifies future maintenance.

7.3 System Architecture

The Smart Attendance System adopts a client–server layered architecture to ensure clear separation of responsibilities, efficient communication between components, and secure management of system data. This architectural approach improves system scalability, maintainability, and flexibility for future enhancements. The architecture comprises three primary layers: the Presentation Layer, the Application Layer, and the Data Layer.

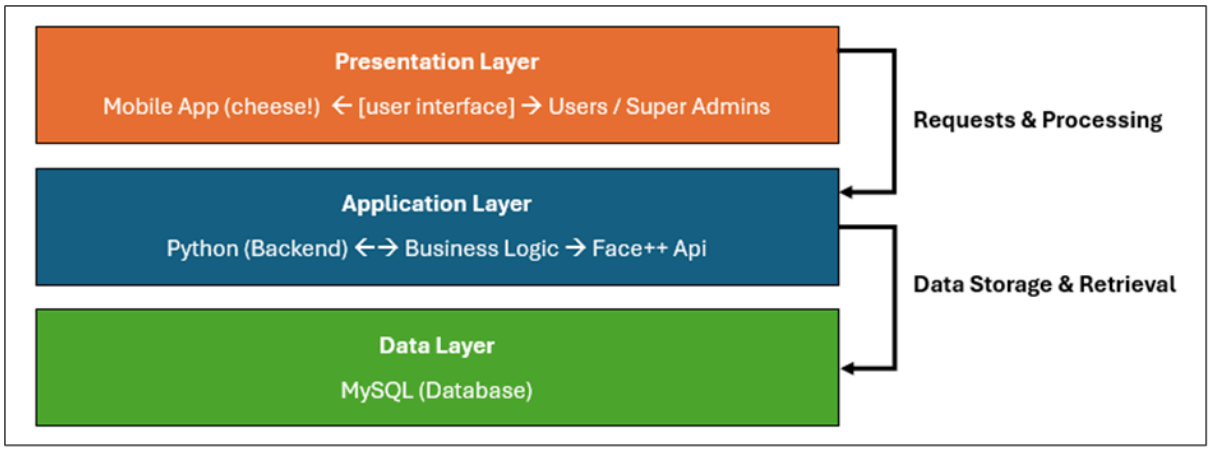

Figure 7.0: Layered Architecture of the Smart Attendance System

Presentation Layer: The presentation layer is implemented through a Flutter-based mobile application that serves as the primary user interface for users and super administrators. This layer handles user interactions and displays system information in an intuitive format. Users can perform various operations, including authentication, student enrolment, attendance recording, report generation, and system configuration.

Application Layer: The application layer serves as the system's core processing component. It is implemented using a Python-based backend that receives and processes requests from the mobile application. This layer is responsible for executing business logic, validating user input, managing attendance workflows, and coordinating system operations. Additionally, the backend integrates with external services such as the Face++ API to perform facial recognition for identity verification during attendance recording.

Data Layer: The data layer is supported by a MySQL relational database that stores and manages all structured system data. This includes records related to attendance, classes, enrolments, logs, student_faces, students, subjects, and users. The database ensures data consistency and integrity, provides secure storage, and enables efficient data retrieval for reporting and analytics.

7.4 Interaction Between Components

System components communicate through RESTful APIs to enable efficient data exchange between the client and server. For instance, when a user captures an attendance photo, the mobile application sends the image to the backend server. The backend then interacts with the Face++ API to perform facial recognition, validates the recognition results, and stores the corresponding attendance records in the database. The processed information is subsequently returned to the client application, allowing the users to view and download attendance reports. Super administrators follow a similar workflow but possess additional privileges for managing users, system logs, and authentication settings.

7.5 Data Flow and Control Mechanisms

The data flow within the system follows a structured sequence to ensure efficient processing and accurate attendance management:

1. Users log in to the system through secure authentication methods, including OAuth 2.0 using Google and Microsoft Single Sign-On (SSO), as well as traditional email and password authentication.
2. Users create classes and schedule attendance sessions.
3. Students are enrolled in the system along with their facial images and personal details.
4. During an attendance session, a photo is captured to record the presence or absence of each student.
5. The backend processes the captured image and validates the faces using the Face++ API.
6. The verified attendance records are then stored in the MySQL database.
7. Attendance reports are generated and displayed within the system, with the option to export them as PDF files.

To ensure system reliability and security, several control mechanisms are implemented. These include backend validation to maintain accurate facial recognition results, role-based access control to restrict sensitive system functions, and encrypted data transmission to safeguard user information. In addition, notification features are integrated to remind users of upcoming attendance sessions, thereby improving system usability and operational efficiency.

7.6 Class Diagram

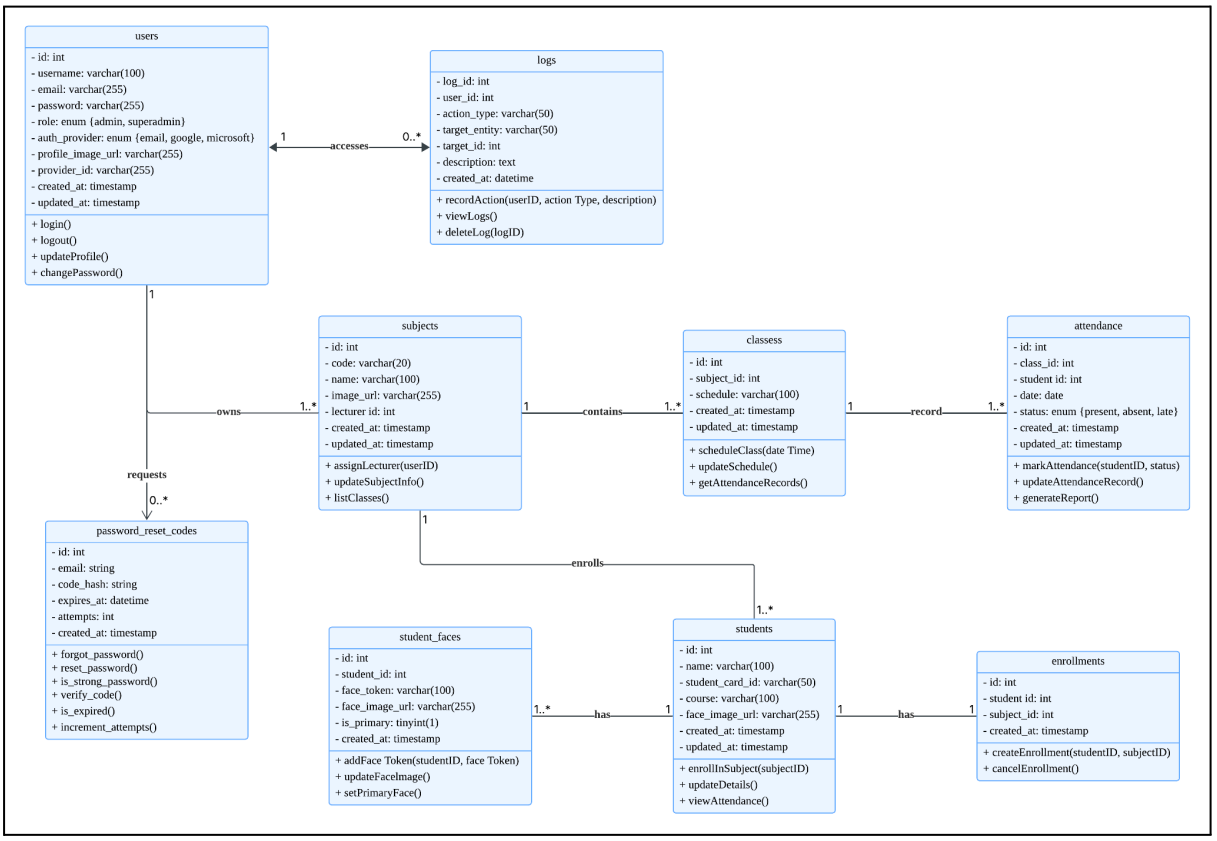

Figure 7.1: UML Class Diagram for Smart Attendance System

The class diagram illustrates the structural design of the student management system by defining its core entities, attributes, operations, and the relationships among them. The Users entity is responsible for managing user authentication and profile information, while the Logs entity records system activities to ensure accountability and traceability. Academic components are represented by the Subjects and Classes entities, which are linked to Attendance records that monitor and track student participation. The Students entity stores detailed information about each learner and is associated with Enrolments, which manage subject registration, and Student_Faces, which store biometric data for facial recognition. A dedicated Password Reset Codes entity facilitates secure account recovery through verification and expiry mechanisms. Each entity includes relevant methods, such as login(), markAttendance(), and createEnrollment(), that represent key system functionalities. Collectively, these entities form an integrated model that supports role-based access control, teaching and learning workflows, attendance tracking, and administrative

management. This structured design ensures that the system effectively meets both functional and security requirements.

7.7 Sequence Diagram

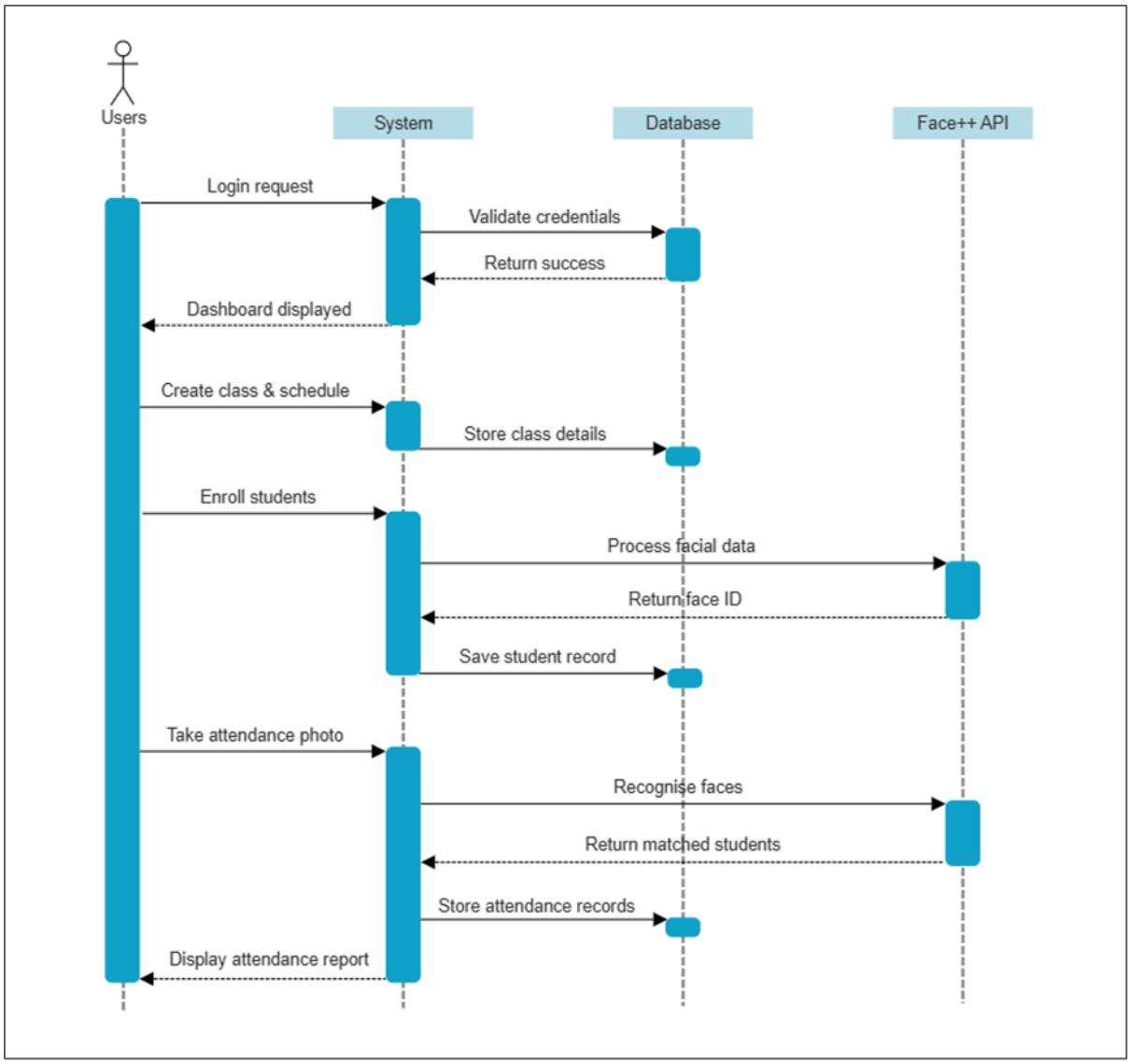

Figure 7.2: Sequence Diagram Illustrating User-System Interaction

The sequence diagram illustrates the dynamic interactions among users, the system, the database, and the external Face++ API during the student attendance process. The process begins when a user submits a login request, which the system verifies against the database before granting access to the dashboard. From the dashboard, lecturers can create classes, schedule attendance sessions, and enrol students. During the enrolment process, students' facial data is captured and sent to the Face++ API for processing. The API then returns a unique face ID, which is subsequently stored in the database for future recognition. When recording

attendance, the system captures a photo of the students and sends it to the Face++ API for facial recognition. The API matches the detected faces with the stored records, and the attendance results are then saved in the database. Finally, the system generates and displays an attendance report to the user. This sequence diagram demonstrates the integration of biometric recognition technology with core academic workflows, highlighting how external services and internal system components collaborate to enable an efficient and automated attendance tracking process.

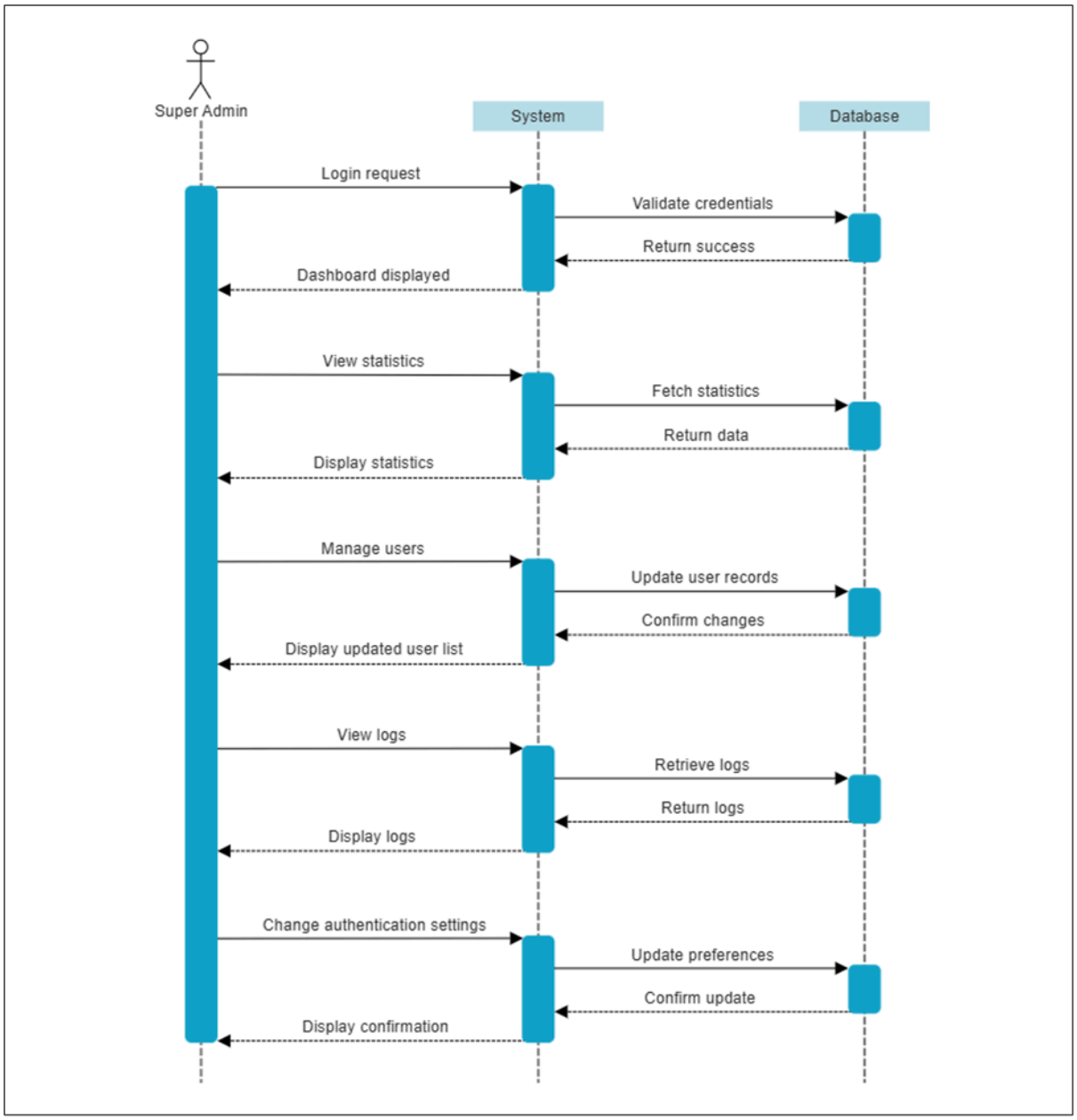

Figure 7.3: Sequence Diagram Illustrating Super Administrator-System Interaction

The sequence diagram illustrates the workflow of a Super Administrator interacting with the system and the database to perform various administrative tasks. The process begins with a login request, during which the system validates the administrator's credentials against the database before granting access to the dashboard. Once authenticated, the Super Administrator can perform several management functions, including viewing system statistics, managing user accounts, accessing system logs, and updating authentication settings. Each action involves the system communicating with the database to retrieve, update, or store relevant information. The system then returns the results or confirmation messages to the Super Administrator through the user interface. This diagram highlights the sequential communication between the user interface, system logic, and database layer, demonstrating how administrative operations are executed in a secure, structured, and efficient manner.

7.8 Activity Diagram

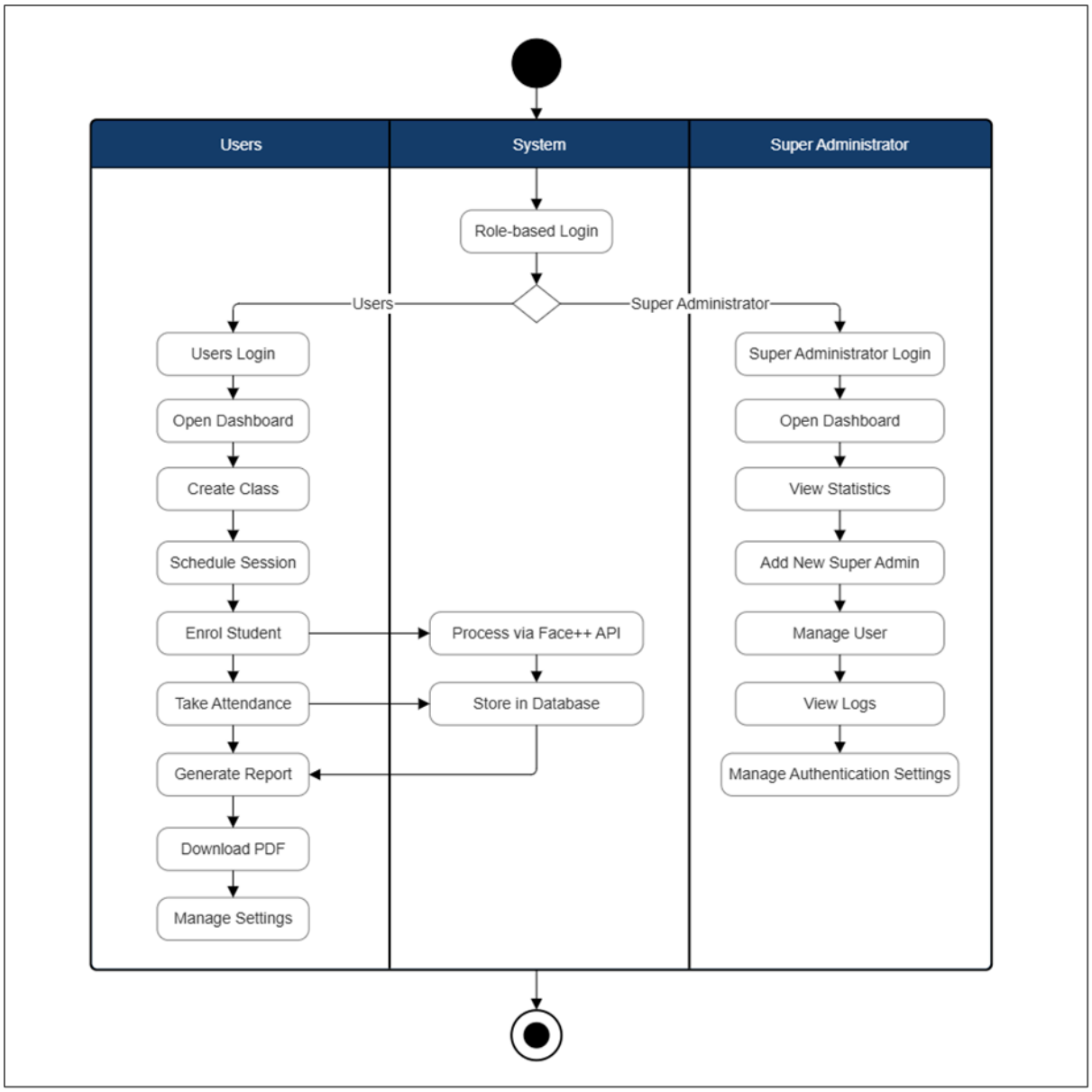

Figure 7.4: Activity Diagram with Swimlanes for the Smart Attendance System

The sequence diagram illustrates the process flow of a role-based login system, demonstrating how both Users and Super Administrators interact with the system. For Users, the workflow begins with the login process and access to the dashboard, followed by tasks such as creating classes, scheduling sessions, enrolling students, recording attendance through the Face++ API, generating attendance reports, downloading reports in PDF format, and managing account settings. For Super Administrators, the diagram highlights their administrative capabilities, including

logging into the system, viewing system statistics, adding new administrators, managing user accounts, accessing system logs, and configuring authentication settings. Each action requires the system to process user requests and either retrieve or store relevant data in the database. This diagram emphasises how different user roles initiate distinct workflows, ensuring that lecturers concentrate on academic and attendance-related tasks, while administrators oversee system management, monitoring, and configuration. As a result, the role-based design enhances both operational efficiency and system security.

7.9 State Diagram

(The combined workflow state diagram for lecturers and super administrators is included in Appendix A.)

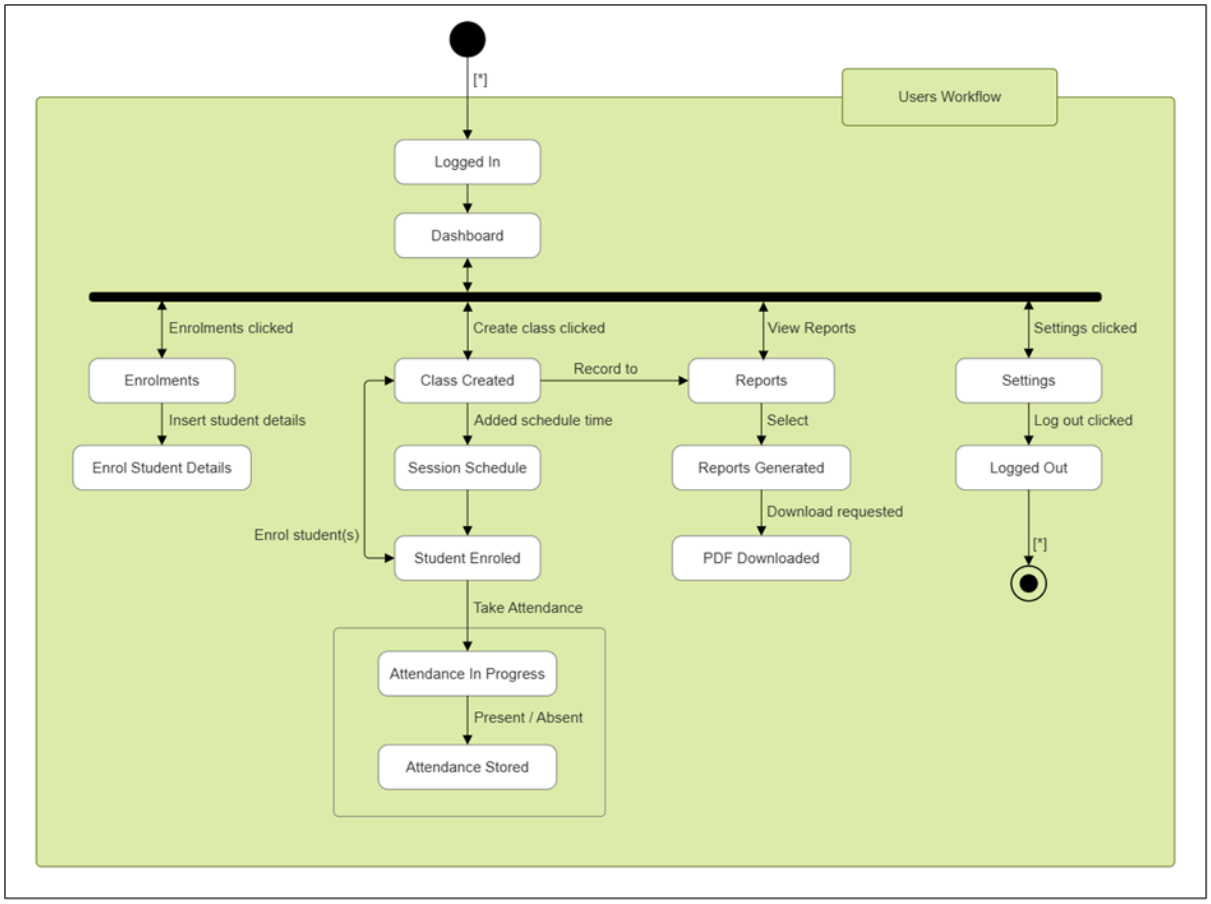

Figure 7.5: Users' Workflow State Diagram for Smart Attendance System

The sequence diagram illustrates the workflow of a user interacting with the system, beginning with the login process and subsequent access to the dashboard. From the dashboard, users can perform several actions, including enrolling students, creating classes, viewing reports, and managing system settings. During the enrolment process, users input student details, which are then processed and stored in the system database. The class creation process enables users to schedule sessions, enrol students into the respective classes, and record attendance. Attendance data is captured and categorised as present or absent, after which the information is stored in the database for future reference. Additionally, users can generate attendance reports and download them in PDF format, while the settings module

allows users to update their preferences or log out of the system. Overall, this diagram highlights the logical sequence of user activities, demonstrating how the system supports key academic operations from student enrolment to attendance tracking and report generation, while also providing essential administrative controls such as settings management and secure logout.

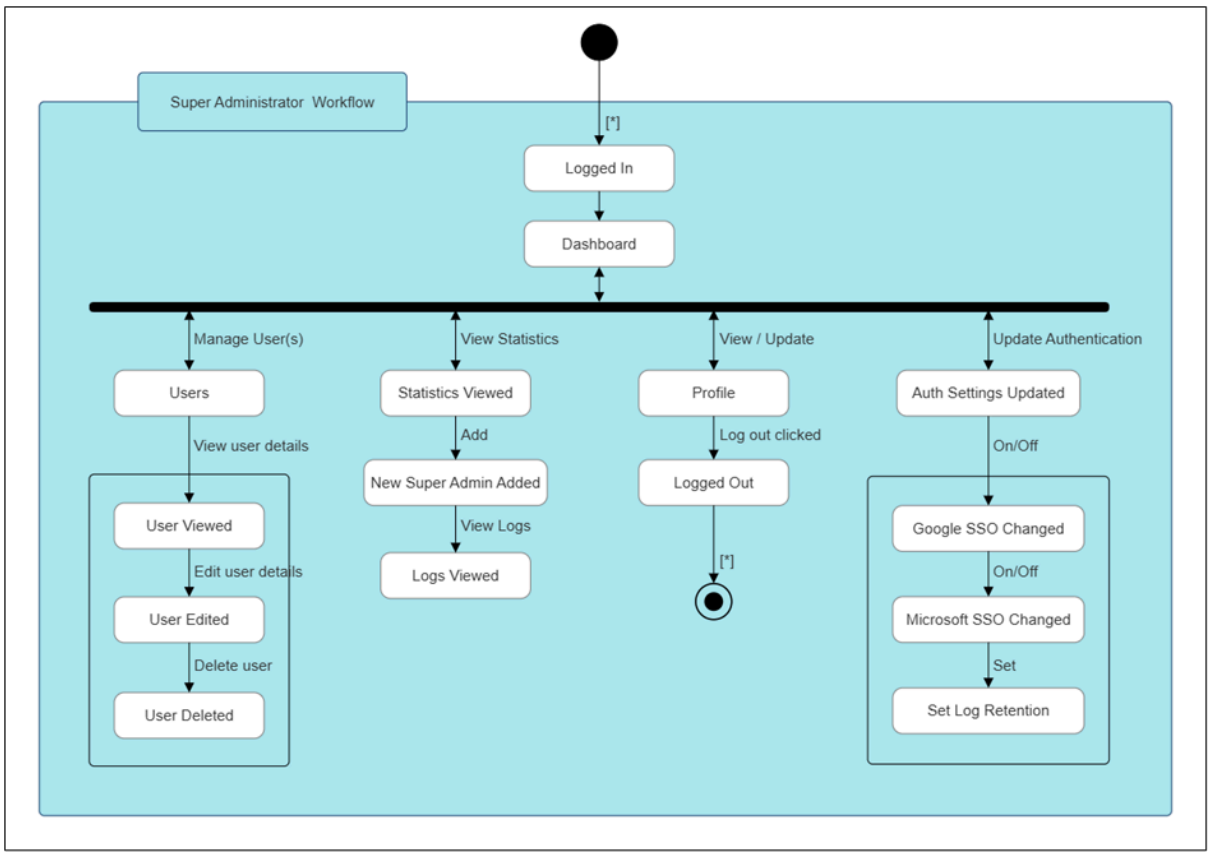

Figure 7.6: Super Administrator Workflow State Diagram for Smart Attendance System

The sequence diagram delineates the activities and decision paths available to a Super Administrator within the system. Following login and access to the dashboard, the Super Administrator can manage users by viewing, editing, or deleting user information. They are also able to view system statistics, add new administrators, and access system logs to monitor activities and ensure accountability. Furthermore, the diagram illustrates the management of authentication settings, including enabling or disabling Google and Microsoft SSO and configuring log retention policies. Additional functionalities include profile management and the ability to securely log out of the system. Overall, this diagram emphasises the administrative scope of the

Super Administrator role, demonstrating how the system facilitates oversight, security, and user management through structured and secure interactions between the user interface, system logic, and database modules.

7.10 Code Quality and Standards

Coding Standards and Conventions

- Consistent naming conventions: snake_case for variables and functions.
- Clear indentation and inline comments to enhance readability and maintainability.

Version Control Practices

- GitHub is used for branching, committing, and merging.
- Structured and descriptive commit messages to ensure clarity of changes.

Code Review and Testing Procedures

- Peer reviews are conducted before merging code into the main branch.
- Unit tests performed on individual modules, including login, enrolment, attendance, reporting, and other related system functions.
- Integration tests to verify API and database communication.

Documentation and Maintainability Practices

- Comprehensive documentation including README files, API references, and system diagrams.
- Documentation ensures ease of maintenance and facilitates future development.

8.0 Quality Assurance

8.1 Quality Assurance Strategy

The Quality Assurance Strategy for the Smart Attendance System outlines a structured approach to ensuring that the system meets defined quality standards and requirements. The main objectives are to verify the system's functional correctness, ensure compliance with non-functional requirements such as performance, security, and usability, and minimise defects through continuous process improvement.

A comprehensive testing strategy is implemented, which includes unit testing, integration testing, system testing, and user acceptance testing. This validation process assesses the system's functionality and reliability at different stages of development. Additionally, quality assurance checklists are used to evaluate critical aspects of the system. Key areas of focus include code quality, consistent naming conventions, sufficient documentation, secure role-based access control implementation, and encrypted storage of facial recognition data. Performance metrics ensure that attendance recognition is processed within acceptable response times, while usability enhancements provide an intuitive dashboard interface and accessible reporting features.

The acceptance criteria state that lecturers must be able to enrol students without encountering system errors, capture attendance images with a minimum facial recognition accuracy of 95%, and reliably generate attendance reports in PDF format. Continuous improvement is supported through GitHub issue tracking, peer code reviews, and Agile retrospectives, which facilitate systematic monitoring and refinement of the development process.

Moreover, the system adheres to internationally recognised software quality standards, particularly ISO/IEC 25010, to ensure maintainability, reliability, and overall software quality. Appropriate data protection principles are implemented to safeguard biometric information and guarantee responsible handling and storage of facial recognition data.

8.2 Quality Assurance Checklist

Table 8.0: Quality Assurance Checklist for System Quality Standards

Category	Checklist Items
Code Quality	● Adherence to consistent naming conventions across all modules. ● Proper indentation and structured formatting to improve code readability and maintainability. ● Comprehensive inline comments to clarify logic and functionality. ● Elimination of unused and redundant code to ensure efficiency and clarity. ● Descriptive GitHub commit messages to accurately document changes and support version control.
Security	● Implementation of role-based access control to manage user permissions effectively. ● Secure encryption and storage of passwords to protect user credentials. ● Encryption of facial recognition data before storage to safeguard biometric information. ● Proper functioning of OAuth 2.0 login (Google/Microsoft SSO) for secure authentication. ● Restriction and verification of super admin privileges to prevent unauthorised access.
Performance	● Attendance photo recognition completed within acceptable time limits. ● Support for up to 15 faces per photo to accommodate multiple students simultaneously. ● Optimised database queries to ensure fast data retrieval and processing. ● Timely generation of reports without delays.

	● Prompt triggering of notifications to ensure immediate communication of attendance updates.
Usability	● Intuitive dashboard navigation for a seamless user experience. ● Clearly labelled enrolment form fields to guide user input. ● Informative error messages are displayed for invalid or incorrect inputs. ● Accessible and prominently displayed attendance button for quick operation. ● Reliable report generation and download in PDF format. ● User-friendly settings page, including profile management, notifications, and password updates.

8.3 Test Plan

(The detailed system test plan is included in Appendix C, Section 5.0.)

The test plan thoroughly validates both the Users Module and the Super Admin Module, ensuring that all core functionalities operate as intended. For the Users Module, test cases cover login scenarios such as successful login, incorrect credentials, missing information, unregistered accounts, and Google or Microsoft SSO. Sign-up scenarios include valid registration, failed attempts, connection errors, and terms not accepted. Dashboard operations tested include class search, group creation, editing and deletion, scheduling, enrolment, attendance tracking, and reporting. Settings management, including profile updates, password changes, reminders, and sign-out, was also verified. All tests were successful, confirming the system's robust handling of both valid and invalid inputs.

For the Super Admin Module, test cases evaluate login processes, dashboard functionalities such as profile viewing, statistics overview, and admin creation, user management including searching, filtering, editing, and deletion, log management through searching and sorting, and system settings including SSO configuration, log retention, and purge options. Each function performed correctly, demonstrating reliable administrative control and effective system oversight.

Overall, the test plan confirms that both modules meet functional requirements, handle errors gracefully, and provide a consistent and reliable user experience across authentication, enrolment, attendance, reporting, and administrative workflows.

9.0 Conclusion

The Smart Attendance Face Recognition System was developed to address the limitations of traditional attendance management methods in educational environments. Conventional techniques such as manual roll calls, sign-in sheets and QR code systems often require significant time during class sessions and may be susceptible to human error or proxy attendance. By integrating facial recognition technology into a mobile application, this projet demonstrates how modern biometric systems can automate attendance recording while improving efficiency, accuracy and transparency.

The system provides a comprehensive attendance management solution through a combination of mobile application interfaces, backend processing services and biometric recognition technology. Lecturers can enrol students, conduct attendance sessions using the device camera and automatically generate attendance reports within seconds. The integration of facial recognition allows the system to verify student identities based on stored facial data, ensuring that attendance is recorded only for individuals who are physically present. In addition, role-based access control allows super administrators to manage system configuration, monitor system logs and oversee user accounts, ensuring secure and organised system administration.

Throughout the development process, the project adopted an Agile methodology that enabled iterative development, continuous testing and progressive system improvement. The system architecture combines a Flutter-based mobile frontend, a Python backend server, a MySQL database for data management and the Face++ API for facial recognition processing. This modular and layered architecture ensures that system components remain maintainable, scalable and adaptable for future enhancements. Quality assurance practices including unit testing, integration testing and usability testing were conducted to ensure that the system meets its functional and non-functional requirements.

The implementation of the Smart Attendance System demonstrates that face recognition technology can significantly reduce administrative workload while improving the reliability of attendance records in educational institutions. The system enables lectures to record attendance quickly without interrupting classroom

activities and provides institutions with accurate data for monitoring student participation.

Although the current system performs effectively in controlled classroom environments, several improvements can be explored in future developments. One potential enhancement is the transition from static image-based recognition to video-based facial recognition, where continuous video streams can be analysed to detect and identify students automatically. Such systems are already widely used in large-scale surveillance and security applications, where recognition can occur even when individuals move or partially obstruct their faces. Implementing a similar approach in an academic environment could improve recognition accuracy, support larger classroom sizes and allow attendance to be recorded dynamically without requiring multiple photographs.

Future improvements may also include integrating the system with institutional learning management systems (LMS), enhancing recognition models to support larger datasets and improving performance under varying environmental conditions such as lighting, seating arrangements and crowd density. These enhancements would further expand the applicability of the system and strengthen its role as part of a modern digital campus infrastructure.

In conclusion, our Smart Attendance Face Recognition System successfully demonstrates how biometric technology and mobile computing can be combined to modernise attendance management in educational institutions. By improving efficiency, reducing errors and providing reliable attendance records, the system contributes to the ongoing digital transformation of academic administrative processes.

10.0 References

Anjum, S., Gondane, S.J., Pipare, M.L. and Valhe, Y.P. (2023) *Student attendance system using QR code - RJPN*. Available at: https://rjpn.org/ijcspub/papers/IJCSP23B1078.pdf (Accessed: 28 January 2026).

Coursera Staff (2023) *What is agile? and when to use it*, *Coursera*. Available at: https://www.coursera.org/in/articles/what-is-agile-a-beginners-guide (Accessed: 26 January 2026).

Coursera Staff (2025) *How to manage project risk: A 5-step guide*, *Coursera*. Available at: https://www.coursera.org/articles/how-to-manage-project-risk (Accessed: 26 January 2026).

Schwaber, K. and Sutherland, J. (2020b) *The 2020 scrum GUIDETM*, *Scrum Guide | Scrum Guides*. Available at: https://scrumguides.org/scrum-guide.html#sprint-planning (Accessed: 26 January 2026).

Appendix A - Integrated State Diagram of Lecturer and Super Administrator Workflows

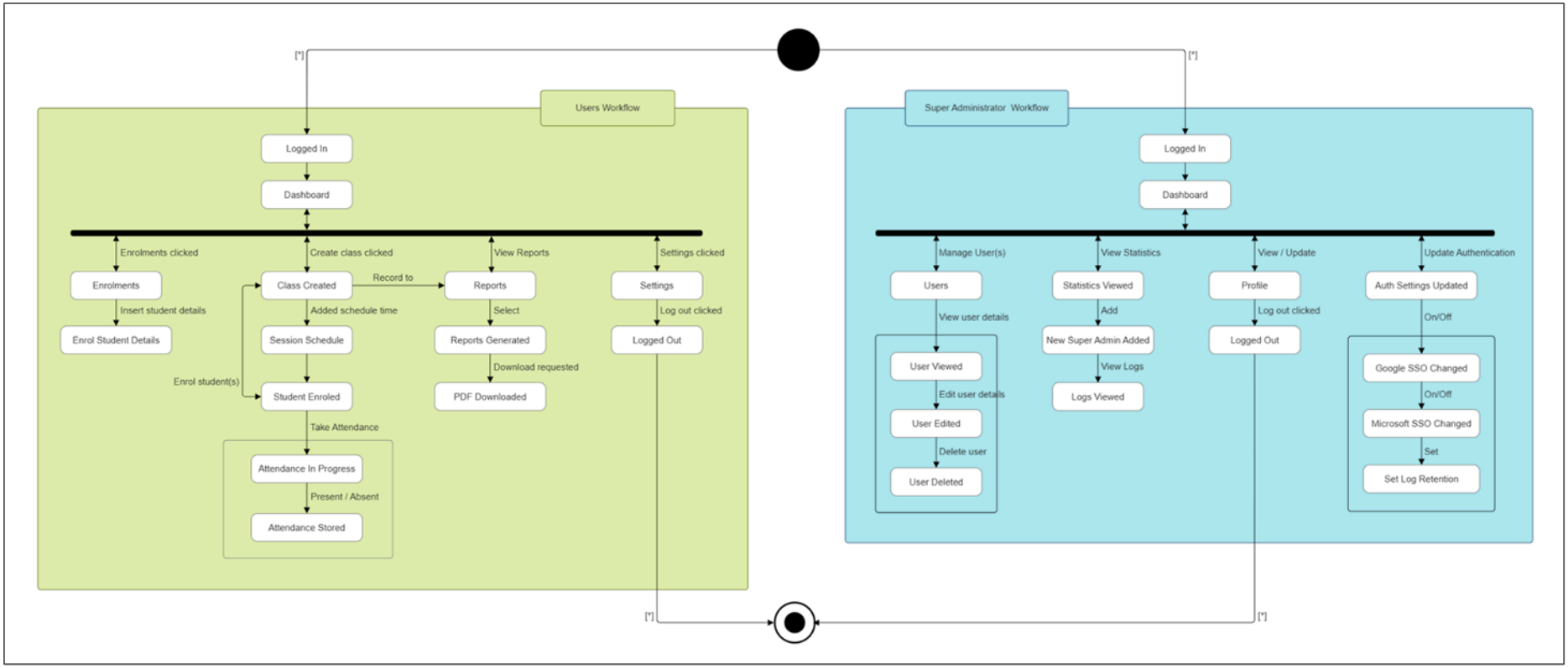

Figure 7.7: Combined Lecturer and Super Administrator Workflow State Diagram for Smart Attendance System

Appendix B

GitHub Link

https://github.com/Joee-eey/Smart-Attendance-System-using-Face-Recognition

YouTube Link

https://youtu.be/DfVLZOofe8c

Presentation Slide Link

https://canva.link/rbqtsfqtwx904h1

Poster Link

https://canva.link/sfjr6ewqanjrvly

Appendix C - UAT Test Plan

1.0 Scope

1.1 Objectives and Business Requirements

The objective of this User Acceptance Testing (UAT) is to evaluate whether the Smart Attendance System using Face Recognition meets the functional and usability requirements for recording student attendance in an academic environment.

Traditional attendance systems rely on manual processes such as paper sign-in sheets or verbal roll calls, which are time-consuming, prone to human error, and susceptible to impersonation. The proposed system introduces an automated attendance recording process using facial recognition technology to improve efficiency, accuracy, and security.

The goals of this UAT are to verify that users can successfully capture student images, process facial recognition, and automatically record attendance within the system.

Success will be measured by evaluating the following criteria:

- The system can correctly recognise registered students using facial recognition.
- Attendance records are automatically generated and stored in the database.
- The system can process multiple student faces in one image capture.
- The time required to record attendance is significantly reduced compared to manual attendance methods.
- Users can easily navigate the system interface without technical difficulties.

If these criteria are achieved, the system will be considered ready for deployment in an educational environment.

1.2 Scope

The purpose of this UAT is to test the core functionality of the Smart Attendance System using Face Recognition, with a focus on the attendance recording process.

The main problem being addressed is the inefficiency and potential inaccuracies of traditional attendance systems. Manual attendance tracking consumes classroom time and allows possible attendance fraud, such as proxy attendance.

For this UAT test, the following aspects will be tested:

- Student face recognition accuracy using uploaded or captured images.
- Ability to detect and recognise multiple students from one or multiple images.
- Automatic attendance recording in the system database.
- Prevention of duplicate attendance records for the same student on the same day.
- Display of attendance records and recognition results within the application.
- Generation of attendance reports for monitoring and reviewing student attendance records.
- Super Admin management functions, including managing administrator accounts and viewing system activity logs.

The following features are not included in this UAT:

- Continuous real-time face recognition through live video streaming. The current system performs facial recognition using captured images from the device camera or uploaded photos rather than real-time video processing.
- Performance and stress testing under large-scale institutional deployment.

This UAT focuses primarily on validating whether the core attendance recognition workflow functions correctly and meets user expectations.

2.0 Testing Team

The User Acceptance Testing will involve several members responsible for coordinating, executing, and documenting the testing process.

Name	**Responsibilities**
Terence Lim Chia Mao	UAT Coordinator – oversees the testing process, manages communication, and monitors testing progress
Lecturer / Supervisor	End User Tester – evaluates system usability and verifies whether the system meets educational requirements
Student Test Participants	Provide facial data and participate in attendance recognition testing
System Developer (Chai Yee Pei, Joey Low Yan Hui, Tan Jun Yee)	Prepare test environment, resolve bugs, and document test results

The testing team will collaborate to ensure the system meets its functional requirements and performs effectively in a real classroom scenario.

3.0 Milestones and Deliverables

This section outlines the testing process, important milestones, and expected deliverables required for successful completion of the UAT phase.

3.1 Design & Testing Process

Before testing begins, the system design and user interface should be finalised to ensure that testers understand how the system operates.

The testing process will be conducted in the following stages:

1. **Staging Environment Setup**
   A testing environment will be prepared that mirrors the production system. The backend server, database, and facial recognition APIs will be configured to ensure the system operates correctly during testing.

2. **Training Session**
   Testers will be introduced to the Smart Attendance System. They will learn how to capture student images, upload photos, and view attendance results.

3. **UAT Execution**
   Testers will perform defined test cases such as capturing group photos, recognising registered students, and verifying attendance records in the system.

4. **Reporting and Evaluation**
   Test results will be collected and analysed to identify any system issues,

usability problems, or improvement opportunities before final deployment.

3.2 Staging Environment

A staging environment will be prepared to simulate the actual system environment without affecting real production data.

The staging system will include:

- A test database containing sample student records.
- Registered student facial data for recognition testing.
- A configured backend server connected to the Face Recognition API.
- The mobile application was installed on test devices.

Testers will access the staging system using designated testing accounts to ensure all activities remain within the testing environment.

3.3 Training

Training sessions will be conducted to ensure testers understand how to operate the system and execute test cases effectively.

The training process will consist of the following sessions:

First Session (10 minutes)
Introduction to the Smart Attendance System and explanation of the project objectives.

Second Session (20 minutes)
Demonstration of system features including:

- Capturing student images

- Uploading multiple images
- Viewing recognition results
- Checking attendance records

Third Session (15 minutes)
Guidelines for executing test cases and reporting bugs or usability issues encountered during testing.

Training ensures that all testers understand their responsibilities and how to properly evaluate the system.

3.4 UAT Execution

The UAT execution phase will take approximately **1 to 3 days**, during which testers will perform various functional test cases.

The testing process will include the following steps:

1. **Tester Onboarding**
   Testers will install the application and gain access to the staging environment.

2. **Test Case Execution**
   Testers will execute predefined test scenarios, including:
   - Capturing student photos using the camera
   - Uploading multiple images from the gallery
   - Detecting multiple faces in group photos
   - Verifying correct student recognition
   - Confirming automatic attendance recording

3. **Bug Reporting and Feedback**
   Any issues encountered during testing will be documented and reported to the developer for analysis and resolution.

3.5 Reporting & Data Analysis

After UAT execution, all test results will be collected and analysed.

The reporting process will include:

- Reviewing test case outcomes.
- Identifying bugs or system errors.
- Evaluating recognition accuracy and system usability.
- Gathering feedback from testers regarding system performance.

Based on the analysis, necessary improvements or bug fixes will be implemented before the system is finalised.

4.0 Environmental Requirements

4.1 Hardware Requirements

To ensure the Smart Attendance System operates efficiently during testing, the following hardware specifications are recommended:

Minimum requirements:

- A smartphone with a functional camera for capturing student images.
- Minimum 4 GB RAM to ensure smooth operation of the mobile application.
- At least 500 MB of available storage for application installation and temporary image storage.
- Stable internet connection for communicating with the backend server and facial recognition API.

Recommended requirements:

- A smartphone with a high-resolution camera for improved facial recognition accuracy.
- Devices with 8 GB RAM or higher for better performance when processing multiple images.
- Reliable Wi-Fi or mobile data connection to ensure fast image upload and recognition processing.

4.2 Software Requirements

The following software components are required for the system to operate during testing:

- Mobile device running the Android operating system
- Smart Attendance System mobile application
- Backend server running Python Flask
- MySQL Database for storing attendance and student data
- Face Recognition API (Face++) for facial detection and identification
- Internet connection for API communication and database access

All software components must be properly configured before UAT execution begins.

5.0 Features To Be Tested

Users Module				
Objectives: To verify that all user, attendance, enrolment, report, and settings functionalities work correctly and meet the expected requirements.				
Test Case	**Description**	**Expected Outcome**	**Actual Outcome (Pass/Failed)**	**User Satisfaction (1 – Unsatisfied, 2 – Neutral, 3 – Satisfied)**
Log in successful	User enters a valid email and password and clicks login	User is successfully logged in and redirected to the dashboard	Pass	3
Log in with an incorrect password.	User enters a registered email and an incorrect password, and clicks the login button	The system displays an error message indicating an incorrect password and prevents login	Pass	3
Log in with missing information	User leaves the email or password field empty and attempts to log in	The system prompts the user to enter the missing information	Pass	3
Log in with an unregistered email.	User enters an email that is not registered in the system	The system displays a message indicating the email is not registered	Pass	3
Log in with Google	User selects the Google login option.	User is authenticated through Google and redirected to the dashboard	Pass	2

Log in with Microsoft	User selects the Microsoft login option	User is authenticated through Microsoft and redirected to the dashboard	Pass	2
Sign up successful	User enters valid details and submits the registration form	Account is created successfully, and the user is redirected to the dashboard/login page	Pass	3
Connection error during sign-up	User attempts to sign up while the connection is interrupted.	The system displays a connection error message.	Pass	2
Sign up failed	User enters invalid or incomplete information	The system displays an error and prevents account creation	Pass	3
Terms not accepted	User submits the sign-up form without accepting the terms and conditions	System prompts the user to accept the terms before proceeding	Pass	3
Dashboard				
Search class	User searches for a class using the search bar	Relevant classes are displayed based on the search keyword	Pass	3
Create a new group	User clicks the create group button and enters group details	The new group has been successfully created and displayed in the dashboard	Pass	3
Enrol members into a group	User clicks the user icon to enrol members into a group	Members are successfully added to the selected group	Pass	3

Edit group details	User edits group name or image through the edit pop-up	Group information is updated successfully	Pass	3
Delete group	User selects delete group	The group is removed from the dashboard upon confirmation in the pop-up	Pass	3
Create a new schedule time	User creates a new schedule time for a group	Schedule time is added successfully	Pass	3
Edit the schedule time	User edits the schedule time using the pop-up	Updated schedule time is saved	Pass	3
Delete schedule time	User deletes a scheduled time	Schedule time is removed from the list	Pass	3
Search for members	User searches for a member within the schedule page	Matching members appear in the results	Pass	3
View attendance statistics	User views the present and absent numbers	System displays the correct total numbers	Pass	3
View recent scan status	User checks recent attendance scans	The system shows the updated attendance status	Pass	3
Sort and filter members	User applies filters (All, Present, Absent, A-Z, Z-A)	Member list updates according to the selected filter	Pass	3

Manually change status	User long-presses a member to change attendance status	Attendance status is updated accordingly	Pass	3
Delete member	User deletes a member from the schedule list	The member is removed from the list successfully after the action is confirmed in the pop-up	Pass	3
Take attendance button	User clicks the take attendance button	Camera interface opens	Pass	3
Camera functions	User uses flashlight, settings, upload image, or swipe camera	The selected camera function operates correctly	Pass	3
Accept multiple photos	User takes multiple photos	The system accepts and processes multiple images for attendance	Pass	3
Image confirmation	User confirms captured image	The image is processed for attendance recognition	Pass	3
Image retake	User retakes the photo	The previous image is discarded, and a new image is captured	Pass	3
Enrolment				
Camera functions	User uses flashlight, settings, upload image, or swipe camera	Camera features work correctly	Pass	3
Accept multiple images	User captures multiple images	The system accepts multiple images for enrolment	Pass	3

Image confirmation	User confirms captured image	The image is saved for enrolment	Pass	3
Image retake	User retakes the photo	The new image replaces the previous one	Pass	3
Enter member details	User navigates to the member details page and enters information	Member details are successfully recorded	Pass	3
Back navigation	User clicks the back arrow	User returns to the previous page	Pass	3
Enrol member	User clicks the enrol button	Member is successfully added to the system	Pass	3
Reports				
Select date	User selects a date using the calendar icon	The system displays attendance data for the selected date	Pass	3
Select group	User chooses a group from the drop-down menu	Report updates based on the selected group	Pass	3
Select schedule time	User chooses a schedule time from the drop-down	Report updates accordingly	Pass	3
View attendance totals	User views the total number of present students	System displays correct numbers	Pass	3
View daily attendance rate	User checks the daily rate percentage	The correct attendance percentage is displayed	Pass	3

View the average attendance rate	User checks the overall average rate	The system calculates and displays the average performance	Pass	3
View performance statistics	User views increase/decrease indicators	Statistics reflect correct performance trends	Pass	2
View line graph	User views the daily statistics graph	The line graph displays attendance trends	Pass	3
Export report	User clicks export report	The system generates a report file	Pass	3
Download PDF	User downloads the report as a PDF	The report is successfully downloaded as a PDF file	Pass	3
Settings				
Choose an existing photo	User uploads an existing profile photo	Profile photo is updated successfully	Pass	2
View profile information	User views username and email	System displays correct user details	Pass	3
Change password	User selects " Change Password " and enters a new password	Password is successfully updated	Pass	2
Attendance reminder toggle	User enables or disables reminders	User enables or disables reminders	Pass	3

Sign out button	User clicks the sign-out button	User is logged out and redirected to the login page	Pass	3

User Feedback: All functions are working as intended.	
Signature: **TS. Choong Hui Ping**	Date: 1/4/2026

Super Admin Module				
Objectives: To verify that all Super Admin functionalities, including login, dashboard management, user management, logs, and settings, operate correctly and meet the expected requirements.				
Test Case	**Description**	**Expected Outcome**	**Actual Outcome (Pass/Failed)**	**User Satisfaction (1 – Unsatisfied, 2 – Neutral, 3 – Satisfied)**
Log in with an incorrect password	Super Admin enters a registered email, but the wrong password, and clicks login	The system shows an error message for an incorrect password and prevents login	Pass	3
Log in with missing information	Super Admin leaves the email or password empty and tries to log in	System prompts user to fill in required fields	Pass	3
Log in with an unregistered email	Super Admin enters an unregistered email	The system shows the “Super Admin account not found” message	Pass	3
Login successful	Super Admin enters a valid email and password	Super Admin is successfully logged in and redirected to the dashboard	Pass	3
Dashboard				

View profile details	Super Admin clicks profile pop-up	Pop-up displays username and email correctly	Pass	3
Change password	Super Admin clicks the " Change Password " button	The password is updated after entering a valid new password	Pass	2
Sign out	Super Admin clicks the sign-out button	User is logged out and redirected to the login page	Pass	3
View total users	Super Admin checks the total user count	The correct total number of users is displayed	Pass	3
View total users' statistics	Super Admin checks the increase/decrease in users	The system shows the correct percentage or trend of user changes	Pass	3
View total admins	Super Admin checks the total admin count	The correct total number of admins is displayed	Pass	3
View total admins' statistics	Super Admin checks the increase/decrease in admins	System shows the correct trend or percentage of admin changes	Pass	3
View total user enrolment	Super Admin checks total user enrolment	The correct number of enrolments is displayed	Pass	3
View total enrolment statistics	Super Admin checks increase/decrease	System shows correct trend/percentage	Pass	2

Add admin button	Super Admin clicks " Add Admin "	An admin pop-up appears with fields: name, email, and password	Pass	3
Add admin pop-up	Super Admin enters valid details and clicks add	New admin is successfully created and visible in the admin list	Pass	3
Users				
Search for users	Super Admin enters a keyword in the search bar	Matching users are displayed	Pass	3
Sort and filter users	Super Admin filters by all/admin/user/email/Google/Microsoft/A-Z/Z-A	User list updates according to the selected filter and sort	Pass	3
Edit user	Super Admin clicks the edit button	Edit pop-up appears with the current user's details	Pass	3
Save user edits	Super Admin updates user details and clicks save	Changes are saved and reflected in the user list	Pass	3
Delete user	Super Admin clicks the delete button	The delete confirmation pop-up appears	Pass	3
Confirm deletion	Super Admin confirms deletion	The selected user is removed from the system	Pass	3
Logs				

Search logs	Super Admin enters a keyword in the search bar	Relevant logs are displayed	Pass	3
Sort logs	Super Admin sorts the logs by default/latest date/earliest date	Logs are reordered according to the selected option	Pass	3
Settings				
Toggle Google SSO	Super Admin enables/disables Google SSO	The setting is saved, and the SSO login reflects the change	Pass	3
Toggle Microsoft SSO	Super Admin enables/disables Microsoft SSO	The setting is saved, and the SSO login reflects the change	Pass	3
Logs retention	Super Admin sets the logs retention period	System saves the retention period and applies it to future logs	Pass	3
Enable auto-purge	Super Admin enables auto-purge toggle	Auto-purge is activated and scheduled as per the retention period	Pass	3
Manual purge	Super Admin clicks “Manual Purge Now” button	Confirmation pop-up appears	Pass	3
Confirm purge	Super Admin confirms purge	Logs are deleted according to the selected purge action	Pass	3

User Feedback: All features are working as intended.	
Signature: **TS. Choong Hui Ping**	Date: 1/4/2026

Appendix D

Flow Chart Link

Boardmix:
https://boardmix.com/app/share/CAE.CKHIoQIgASoQYNtWksCNQdRsmZA-dm2WeDAGQAE/YYISPw